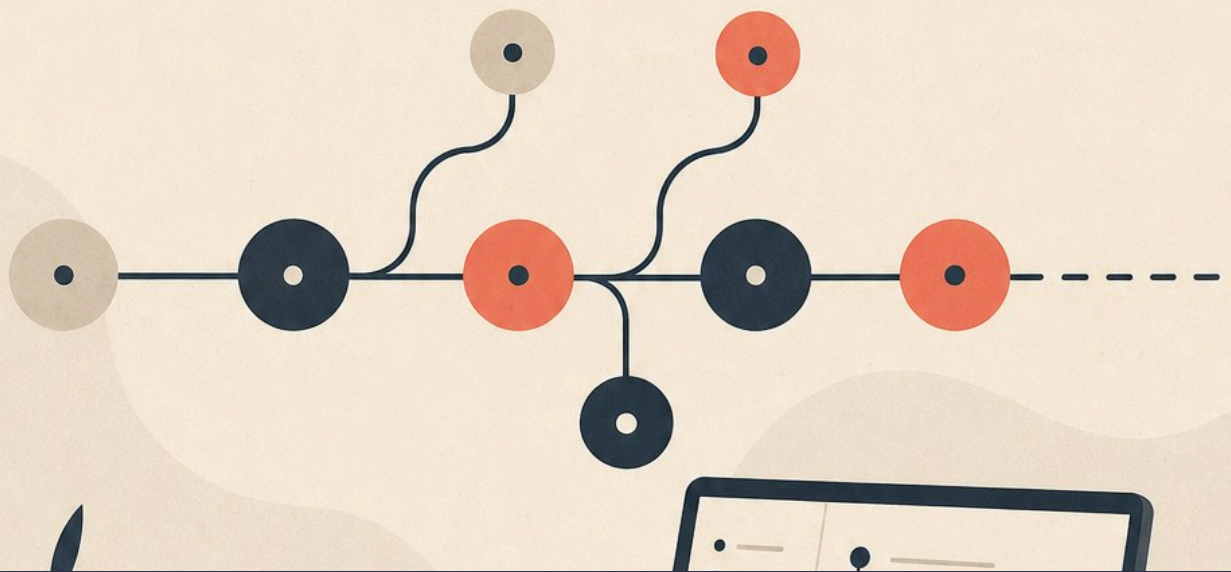


Formation débutant



FORMATION INFORMATIQUE

Git

Garder l'historique de son code, collaborer
sans se marcher dessus.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Salut, c'est quoi Git ?

- [Ce que ce n'est pas](#)
- [Ce que tu vas savoir faire](#)
- [Comment lire ce livre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

2. Chapitre 2 - Installer Git

- [Ce que ce n'est pas](#)
- [Windows et verification](#)
- [Premiere config \(obligatoire\)](#)
- [Ou taper les commandes ?](#)
- [Aide integree](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

3. Chapitre 3 - Ton premier depot

- [Ce que ce n'est pas](#)
- [Creer, initialiser, premier fichier](#)
- [git init dans un projet existant](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

4. Chapitre 4 - add et commit (la photo)

- [Ce que ce n'est pas](#)
- [Ajouter et committer](#)
- [Voir que ca a marche, puis continuer](#)
- [Amend \(aperçu, avec prudence\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

5. Chapitre 5 - Lire l'historique

- [Ce que ce n'est pas](#)
- [log et diff](#)
- [Checkout / restore d'un fichier \(aperçu\)](#)
- [Petite histoire](#)

- Erreur classique
- En vrai
- A toi

6. Chapitre 6 - Les branches

- Ce que ce n'est pas
- Creer, changer, travailler
- A quoi ca sert vraiment ?
- Renommer / supprimer
- Petite histoire
- Erreur classique
- En vrai
- A toi

7. Chapitre 7 - Fusionner (merge)

- Ce que ce n'est pas
- Scenario de base
- Apres le merge
- Quand ne pas merger directement ?
- Merge vs rebase (aperçu)
- Petite histoire
- Erreur classique
- En vrai
- A toi

8. Chapitre 8 - Les conflits

- Ce que ce n'est pas
- Comment ca arrive et a quoi ca ressemble
- Resolving
- Annuler un merge en cours
- Eviter les conflits (un peu)
- Petite histoire
- Erreur classique
- En vrai
- A toi

9. Chapitre 9 - .gitignore

- Ce que ce n'est pas
- Creer le fichier
- Verifier et reparer
- Patterns utiles et templates
- Petite histoire
- Erreur classique
- En vrai
- A toi

10. Chapitre 10 - GitHub, c'est quoi ?

- Ce que ce n'est pas

- [Compte et nouveau depot](#)
- [Lier ton depot local](#)
- [SSH ou HTTPS ?](#)
- [README.md](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

11. [Chapitre 11 - clone, push, pull](#)

- [Ce que ce n'est pas](#)
- [clone, push, pull](#)
- [Erreurs frequentes](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

12. [Chapitre 12 - De bons messages de commit](#)

- [Ce que ce n'est pas](#)
- [Oui et Non \(exemples concrets\)](#)
- [Corps optionnel et Conventional Commits](#)
- [Exemple de fil propre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

13. [Chapitre 13 - Mini-projet : carnet versionne](#)

- [Ce que ce n'est pas](#)
- [But](#)
- [Etapas](#)
- [Variante avantee](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

14. [Chapitre 14 - Ce qu'il faut retenir \(bases\)](#)

- [Ce que ce n'est pas](#)
- [Boucle quotidienne](#)
- [Commandes vitales](#)
- [Zones](#)
- [Habitudes](#)
- [Erreurs classiques \(recap\)](#)
- [Aide-memoire express](#)
- [Petite histoire](#)

- Suite immediate
- A toi

15. Chapitre 15 - Stash et annuler (avec prudence).

- Ce que ce n'est pas
- stash
- restore et reset (aperçus).
- revert (plus sûr en équipe).
- Petite histoire
- Erreur classique
- En vrai
- A toi

16. Chapitre 16 - Pull requests (PR).

- Ce que ce n'est pas
- Scenario type
- Pourquoi pas merger en silence ?
- Bonne description
- But
- Changes
- Test
- Merge buttons
- Petite histoire
- Erreur classique
- En vrai
- A toi

17. Chapitre 17 - Atelier : lire les messages Git

- Ce que ce n'est pas
- Traductions utiles
- Méthode DanielCraft
- Exercice 1 - Ahead / behind (10 min).
- Exercice 2 - Rejected (10 min).
- Exercice 3 - Conflit (10 min).
- Exercice 4 - Secret (8 min).
- Petite histoire
- Erreur classique
- En vrai
- A toi

18. Chapitre 18 - Atelier : workflow d'équipe (simule).

- Ce que ce n'est pas
- Idee et tour de jeu
- Règles d'équipe simples
- force push ?
- Protection de branche
- Mini charte (à coller).

- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

19. [Chapitre 19 - Atelier : reparer sans paniquer](#)

- [Ce que ce n'est pas](#)
- [Checklist anti-stress](#)
- [Cas 1 - Mauvais fichier stage](#)
- [Cas 2 - Mauvais message \(pas encore push\)](#)
- [Cas 3 - Commit en trop \(pas push\)](#)
- [Cas 4 - Commit déjà push \(equipe\)](#)
- [Cas 5 - Mauvaise branche](#)
- [Cas 6 - Conflit qui te dépasse](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Exercice final atelier](#)
- [En vrai](#)
- [A toi](#)

20. [Quiz final - Teste-toi !](#)

- [Ce que ce n'est pas](#)
- [Questions](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Reponses](#)
- [Score](#)
- [En vrai](#)
- [A toi](#)

21. [Bravo.](#)

- [Ce que ce n'est pas](#)
- [Ce que tu sais faire](#)
- [Mission finale](#)
- [La suite \(quand tu voudras\)](#)
- [Rituel des 2 minutes](#)
- [Engagement de 30 jours](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

Chapitre 1 - Salut, c'est quoi Git ?



Git garde l'historique de ton projet, version apres version.

Git, ce n'est pas un reseau social pour developpeurs, et ce n'est pas non plus une baguette magique qui "sauvegarde tout seul" dans le nuage. C'est un outil pour garder l'**historique** de ton projet : qui a change quoi, quand, et pourquoi. Chaque "photo" s'appelle un **commit**. Tu modifies des fichiers. Tu dis a Git : "garde cette version". Plus tard, tu peux revenir en arriere, comparer deux moments, essayer une idee sur une **branche** parallele, ou travailler a plusieurs sans t'ecraser mutuellement. En 2026, quand une equipe dit "c'est sur le repo", elle parle presque toujours d'un **depot** Git, souvent heberge aussi sur GitHub ou GitLab.

Chez DanielCraft, on aime une image nette. Ton projet, c'est un album de photos. Chaque commit, c'est une photo datee avec un petit titre. Les branches, ce sont des albums paralleles : tu peux tester une mise en page risquee sans toucher a la version stable. Lea versionne ses sites clients et retrouve un CSS ecrase en trente secondes. Max versionne un carnet de devis pour ne plus perdre ses calculs. Sam fait versionner les exercices de ses eleves pour montrer qui a change quoi. Trois metiers, meme logique : ne plus perdre le fil, oser experimenter, collaborer sans panique.

Git ne remplace pas ta tete. Il ne decide pas si ton code est bon. Il enregistre ce que tu choisis d'enregistrer. Si tu commit un secret ou un fichier casse, Git le gardera aussi - proprement, avec une date. C'est pour ca qu'on apprend le `.gitignore` et les messages clairs. L'outil est puissant parce qu'il est honnete : il fait ce que tu lui demandes, pas ce que tu voulais dire en pensant fort.

★ A RETENIR

Git photographie ton travail. Toi, tu choisis quand declencher, sur quelle branche, et ce que tu fusionnes.

Ce que ce n'est pas

Ce n'est pas GitHub. Git, c'est le logiciel sur ton ordi. GitHub, c'est un site qui heberge des depots Git en ligne et ajoute la collaboration (issues, pull requests, review). Tu peux utiliser Git tout seul, sans compte, dans un dossier local. GitHub devient utile quand tu veux partager, sauvegarder au loin, ou travailler a plusieurs. Ce n'est pas non plus "trop complique pour un debutant". Les cinq commandes du quotidien -

status, add, commit, log, push - suffisent déjà à être utiles. Le reste vient après, quand tu as un vrai problème à résoudre.

Ce n'est pas non plus un disque dur magique dans le nuage. Tant que tu n'as pas poussé ailleurs, l'historique vit surtout chez toi. Utile pour travailler. Pas suffisant comme seule sauvegarde lointaine si ton disque lâche. Et ce n'est surtout pas une excuse pour committer des secrets : mots de passe, clés API, fichiers `.env`. On y reviendra avec `.gitignore`.

Ce que tu vas savoir faire

Dans ce livre, tu vas installer Git et configurer ton nom et ton email. Tu créeras un dépôt avec `git init`. Tu manipuleras `add`, `commit`, `status` et `log`. Tu feras des branches, des fusions et des conflits. Tu écriras un `.gitignore`. Tu cloneras, pousseras et tireras depuis GitHub. Tu tapoteras stash et les annulations simples. Tu liras et ouvriras une pull request. Puis un mini-projet, un recap, trois ateliers, un quiz, et un bravo.

Niveau débutant solide. Pas besoin d'être "dev senior". Besoin de curiosité, de taper les commandes dans un vrai dossier, et d'accepter de lire les messages que Git t'envoie au lieu de cliquer au hasard.

Comment lire ce livre

Lis dans l'ordre au début. Les premiers chapitres posent le sol : init, add, commit, branches. Les ateliers font faire avec les mains. Le quiz vérifie sans humilier. Tu peux revenir ensuite à un chapitre précis (conflits, stash, PR) comme à une fiche. À chaque fin de chapitre, il y a un "À toi". Fais-le. Cinq minutes valent mieux qu'une lecture passive. Git s'apprend en cassant volontairement, puis en réparant. Casse sur `mes-tests-git`, pas sur le projet client du vendredi soir.

♦ ASTUCE

Crée un dossier `mes-tests-git` dès le chapitre 2. Tout ce livre s'y joue. Tu peux casser sans paniquer.

Petite histoire

Lea a écrasé un fichier CSS important un jeudi soir. Avant Git, c'était la panique et une sauvegarde "final_v3_vrai_definitif". Avec Git, elle a ouvert `git log --oneline`, retrouve le commit d'avant, compare avec `git diff`, respire, restaure. Vingt minutes au lieu de deux heures de stress. Max a voulu essayer une mise en page risquée sur une branche `feature/nouveau-header`. Ça n'a pas marché. Il a jeté la branche. `main` était intact. Sam a montré un conflit en classe : les élèves ont vu les marqueurs `<<<<<<`, discute, choisis, commit. Le conflit est devenu un exercice, pas un drame. Chez DanielCraft, c'est exactement le but : rendre l'erreur réversible et la collaboration visible.

Erreur classique

Croire que "Git, c'est trop compliqué pour moi". Ou croire que "je vais tout casser dès que je tape une commande". Souvent, le problème n'est pas Git. C'est l'absence de terrain de jeu : tu testes sur un projet critique la veille d'une livraison. Crée `mes-tests-git`. Casse-la. Repars. Autre piège : tout apprendre d'un coup (rebase interactif, hooks, sous-modules) avant de maîtriser status, add, commit, log. Commence petit. Monte ensuite. Encore un piège : confondre "j'ai Git" et "mon travail est sauvegardé". Sans push ou backup, tu restes dépendant d'un seul disque.

⚠ ATTENTION

Git enregistre ce que tu lui demandes. Secret committe = secret dans l'historique. On apprend `.gitignore` tot dans le livre.

En vrai

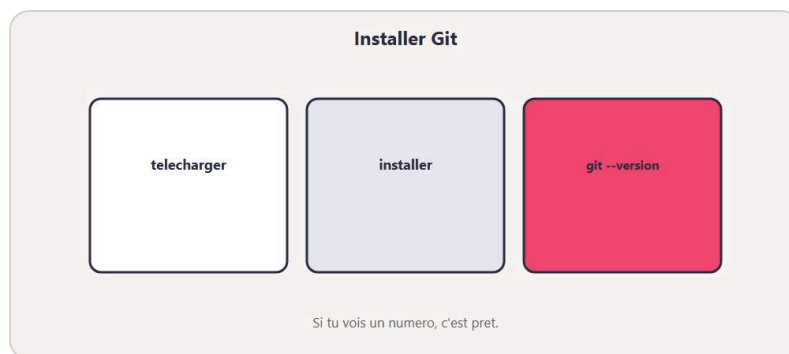
Dis a voix haute : "Je veux une machine a photos de mon travail". C'est Git. Ouvre un terminal si tu peux. Si `git --version` repond deja, note le numero quelque part. Sinon, le chapitre suivant installe. Tu n'as rien a prouver a personne. Tu as juste a commencer dans un dossier de test.

Une sauvegarde cloud copie des fichiers. Git raconte une histoire : qui, quand, pourquoi, quoi. Tu peux comparer deux moments precis. Tu peux isoler une experience sur une branche. Ce n'est pas la meme promesse. Chez DanielCraft, on garde les deux idees distinctes : sauvegarder n'est pas versionner, versionner n'est pas publier.

A toi

Ecris 3 peurs ou doutes sur Git (ex : "j'ai peur d'effacer", "je ne comprends pas les branches"). Garde ce papier. On y repondra dans le livre. Puis ecris une tache concrete que tu aimerais versionner : site web, notes de cours, scripts Python, carnet de devis. Ce sera ton fil rouge.

Chapitre 2 - Installer Git



Telecharger, installer, tester avec `git --version`.

Avant de photographier un projet, il te faut Git qui repond vraiment. Pas une icone dans le menu demarrer. Une commande `git --version` qui affiche un numero, dans un vrai **terminal**, sur ta machine. Sur Windows, le chemin simple reste `git-scm.com` : tu telecharges l'**installateur**, tu suis l'assistant (les options par default vont bien au debut), tu ouvres un terminal, tu verifies. Si un numero apparait, c'est bon. Sans ca, tout le reste du livre reste theorique, et la theorie ne sauve pas un fichier ecrase un jeudi soir.

Chez DanielCraft, on traite aussi la config comme obligatoire. Git a besoin de savoir qui tu es pour signer tes commits. Chaque photo de l'album portera ton nom et ton email. Lea a oublie une fois et s'est retrouvee avec une identite bizarre dans l'historique. Max a configure avec l'email de son futur compte GitHub des le premier jour. Sam verifie la config au tableau avant le premier commit de la classe. Toi, tu vas faire pareil avant de toucher `add` ou `commit`.

Installer, c'est aussi choisir ou tu tapes. PowerShell, Git Bash, terminal integre de VS Code : les commandes de ce livre restent les memes. Ce qui change, c'est le confort. Au debut, reste sur les messages de commit en une ligne avec `-m "..."`. Plus simple que de te battre avec un editeur vi ouvert par surprise parce que tu as oublie le `-m`.

★ A RETENIR

Sans `git --version` et sans `user.name` / `user.email`, tu n'es pas pret. Configure avant le premier commit, pas apres.

Ce que ce n'est pas

Installer Git, ce n'est pas creer un compte GitHub. Tu peux versionner en local des semaines sans compte en ligne. Ce n'est pas non plus tout comprendre de l'univers DevOps. Ce n'est pas obligatoire d'ouvrir vingt options avancees de l'installateur Windows : les defaults suffisent pour apprendre. Et ce n'est surtout pas "optionnel" de renseigner `user.name` et `user.email`. Sans ca, Git rale, ou pire, enregistre une fausse identite que tu regretteras quand tu pousseras sur un projet d'equipe.

Ce n'est pas non plus "terminer le livre d'un coup". Ce chapitre est un sas. Tu installes. Tu configures. Tu crees un dossier de test. Puis tu passes au depot.

Windows et verification

1. Va sur `git-scm.com`

2. 2. Telecharge l'installateur
3. 3. Suis l'assistant (defaults OK au debut)
4. 4. Ouvre un terminal et tape :

```
git --version
```

Si un numero s'affiche, l'appareil est la. Note-le quelque part dans un fichier `notes.md` si tu veux. Ce n'est pas pour briller. C'est pour savoir quoi dire si un jour un outil demande "quelle version de Git ?".

♦ ASTUCE

Git Bash, PowerShell ou le terminal VS Code : les commandes sont les memes. Choisis celui ou tu te sens le plus a l'aise et garde-le une semaine.

Premiere config (obligatoire)

```
git config --global user.name "Ton Prenom"  
git config --global user.email "ton@email.com"
```

Verifie :

```
git config --global --list
```

Utilise un email que tu assumeras (souvent celui de ton compte GitHub). Ce n'est pas de la paperasse. C'est ta **signature** sur l'historique. Lea a mis son email pro. Max a mis celui de son compte GitHub avant meme de creer le compte - il a juste veille a rester coherent. Sam exige l'email "officiel" des eleves pour que les commits soient lisibles en correction.

⚠ ATTENTION

Sans `user.name` et `user.email`, Git enregistre une fausse identite. Configure avant ton premier commit, pas apres.

Ou taper les commandes ?

Tu peux utiliser le terminal Windows (PowerShell), Git Bash (installe avec Git), ou le terminal integre de VS Code. Les commandes de ce livre marchent dans ces environnements. Si `git` n'est pas reconnu juste apres l'install, ferme et rouvre le terminal : le PATH se met a jour au demarrage. Si ca bloque encore, redemarre une fois. Chez DanielCraft, on aime ces petites verites de terrain plus que les theories parfaites.

Aide integree

```
git help  
git help commit  
git commit -h
```

Tu n'as pas à tout lire. Tu dois savoir que l'aide existe. "Je ne sais pas" suivi de `git help` bat "j'invente une commande" neuf fois sur dix. Lea ouvre `git help commit` quand elle oublie un flag. Max a commencé à le faire après avoir tapé trois fois une fausse option. Sam projette l'aide au tableau pour montrer que Git n'est pas un mur opaque.

Petite histoire

Lea a installé Git un lundi matin, configuré, ouvert VS Code, tapé `git --version` dans le terminal intégré. Sensation : "l'outil est là". Elle a créé aussitôt un dossier `mes-tests-git` sur le bureau, pas dans Documents/ProjetsClients. Terrain de jeu séparé. Max a fait pareil le soir même, en notant ses deux commandes de config dans un carnet papier. Sam refuse de démarrer un atelier sans cette vérification collective : `git --version` à voix haute, puis config, puis dossier de test. Trois minutes qui évitent une heure de "chez moi ça marche pas". Chez DanielCraft, c'est exactement le genre de cérémonie ridicule qui sauve des soirées.

Erreur classique

Tu commit sans avoir configuré le nom et l'email. Git rale. Ou pire : il utilise une fausse identité que tu pousseras ensuite sur GitHub. Configure tout de suite. Autre classique : croire que l'install a échoué parce que tu es dans le mauvais terminal - reouvre, retape, vérifie le PATH si besoin. Encore un piège : installer Git et passer directement à un projet client critique. Non. Crée `mes-tests-git`. Casse-la. Apprends-la. Autre erreur : vouloir "tout configurer" (éditeur par défaut, aliases, credential helper avancé) avant d'avoir fait un seul commit. Trop tôt. Les défauts suffisent.

En vrai

Ouvre VS Code. Ouvre le terminal. Tape `git --version`. Si ça marche là, tu es prêt pour la suite. Crée aussi le dossier `mes-tests-git` sur ton disque : c'est ton terrain de jeu pour tout le livre. Écris dedans un petit `notes.md` avec la date et la version de Git. Tu viens de commencer ton album avant même le premier vrai commit.

A toi

1. Lance `git --version`.
2. Configure `user.name` et `user.email`.
3. Affiche la config avec `git config --global --list`.
4. Crée `mes-tests-git`.

Note le tout dans un fichier `notes.md` si tu veux garder une trace. Demain matin, avant le chapitre 3, retape juste `git --version` pour sentir que l'outil est toujours là. Cinq secondes. Habitude posée.

Chapitre 3 - Ton premier depot



git init : le depot commence dans ton dossier.

Un **depot** (repository), c'est un projet suivi par Git. Pas une usine. Pas un compte en ligne. Un dossier + une memoire cachee **.git**. Tu initialises. Tu ajoutes des fichiers. Tu regardes **status**. Tu comprends les **zones**. A partir de la, chaque chapitre ajoute une commande utile. Mais le sol, c'est ici : un depot propre, dans le bon dossier, avec un premier fichier. Sans ce sol, les branches et les push ne sont que du jargon.

Chez DanielCraft, on insiste sur "le bon dossier" parce que le classique du debutant, c'est **git init** dans le dossier utilisateur entier. Lea l'a failli faire un soir de fatigue. Max verifie toujours avec **cd** avant de taper **init**. Sam fait dire a voix haute le chemin complet. Une ceremonie ridicule qui evite une galere reelle : un **.git** qui essaie de suivre des milliers de fichiers personnels. Toi, avant chaque **init**, tu demandes : "suis-je bien dans le dossier du projet, et seulement de ce projet ?"

Quand **git init** reussit, presque rien ne change a l'oeil nu. Pourtant quelque chose d'immense vient d'arriver : Git a allume la machine a photos dans ce dossier. Tu ne le vois pas tant que tu n'actives pas "elements caches" dans l'explorateur. Le dossier **.git** est la. Ne le touche pas a la main. C'est la memoire. Si tu le supprimes, tu perds l'historique local.

Trois zones structurent tout le reste du livre. Le **dossier de travail** : tes fichiers normaux, ceux que tu ouvres dans l'editeur. L'**index** / staging : ce que tu prepares pour la photo, apres **add**. Le depot : l'historique des commits, dans **.git**. Tu ecris dans tes fichiers. Tu choisis quoi photographier (**add**). Tu valides la photo (**commit**). Beaucoup de debutants confondent index et historique. **status** te le dit clairement si tu apprends a le lire : untracked, staged, modified, clean.

★ A RETENIR

git init allume la machine a photos dans ce dossier. Ce n'est ni un commit, ni une publication en ligne.

Ce que ce n'est pas

git init, ce n'est pas publier sur internet. Ce n'est pas un commit. Ce n'est pas "sauvegarder dans le cloud". C'est allumer la machine a photos dans ce dossier. Point. Et ce n'est surtout pas une invitation a fouiller **.git** a la main : ne le touche pas. Ce n'est pas non plus **git clone** : clone copie un depot existant depuis ailleurs ; init demarre un depot neuf ici. Les deux menent a un dossier avec un **.git**, mais le chemin n'est pas le meme.

Ce n'est pas non plus "finir le projet". Un depot vide avec un readme est deja un debut propre. Lea initialise parfois avant d'avoir ecrit une ligne de CSS. Max initialise des qu'il a un carnet de notes. Sam initialise avec la classe des qu'un exercice commence. Habitude : allumer tot, photographier souvent.

Creer, initialiser, premier fichier

```
mkdir mon-carnet
cd mon-carnet
git init
```

Git cree un dossier cache `.git`.

Cree `readme.txt` avec :

```
Mon carnet de tests Git
```

Puis :

```
git status
```

Git dit souvent quels fichiers sont non suivis (untracked), modifies, ou prêts a commit. Lis `status` souvent. C'est ton tableau de bord. Lea le lance presque apres chaque geste. Max le lance quand il a un doute. Sam le lance avant de repondre a un eleve. Chez DanielCraft, on prefere dire `status` de trop qu'un commit a l'aveugle.

♦ ASTUCE

Lis `git status` a voix haute apres chaque geste. En une semaine, les mots untracked, staged et clean deviennent familiers.

git init dans un projet existant

Tu peux lancer `git init` dans un dossier qui a deja des fichiers. Puis tu choisis quoi ajouter. Utile pour "mettre sous Git" un site ou un script deja commence. Verifie quand meme que tu n'es pas dans un mega-dossier plein de secrets, de telechargements, ou de `node_modules` enormes. Si le dossier est sale, nettoie ou ignore avant le premier gros `add`. On detaille `.gitignore` plus loin. Ici, retiens juste : init dans le bon endroit, avec les bons fichiers sous les yeux.

Petite histoire

Max a active "elements caches" dans l'explorateur Windows. Apres `init`, `.git` est apparue. Sensation concrete : "il s'est passe quelque chose". Sam a fait ajouter `notes.txt`, relancer `status`, comparer avec avant. Les eleves ont vu le mot untracked. Lea a lu a voix haute : "untracked files". Chez DanielCraft, on celebre ce moment : tu vois le langage de Git avant de le commander.

Lea, le meme soir, a initialise un second dossier par erreur dans le mauvais chemin. Elle a vu des centaines de fichiers untracked. Elle a compris. Elle a ferme. Elle a recommence dans `mes-tests-git`. Lecon gratuite. Toi, si `status` te montre la moitie de ton disque, stop. Tu es au mauvais endroit.

Erreur classique

`git init` dans le mauvais dossier (ex : ton dossier utilisateur entier). Verifie avec `cd` et `pwd` (ou `cd` sous PowerShell) avant. Autre piege : paniquer parce que `status` "parle anglais". Normal. Apprends les mots : untracked, staged, nothing to commit. Encore un piege : supprimer `.git` "pour nettoyer" sans comprendre que tu effaces l'historique. Si tu veux recommencer un depot de test, oui, tu peux supprimer `.git` dans `mes-tests-git`. Sur un projet client, jamais sans savoir ce que tu fais.

⚠ ATTENTION

Ne touche jamais le dossier `.git` a la main. C'est la memoire de Git. Si tu le supprimes, tu perds l'historique local.

En vrai

Ouvre l'explorateur. Active "elements caches". Tu dois voir `.git` apparaitre apres `init`. Ajoute un second fichier `notes.txt`. Relance `status`. Compare avec avant. Dis a voix haute ce que Git sait maintenant de ton dossier. Si tu peux le dire simplement, tu as compris le chapitre.

Imagine une chaine. Tu ecris dans tes fichiers (zone de travail). Tu choisis quoi photographier (`add` = index). Tu valides la photo (`commit` = historique). Si `status` dit "Changes to be committed", c'est pret pour la photo. Si ca dit "Changes not staged", il manque un `add`. Si ca dit "nothing to commit, working tree clean", tu es a jour. Cette lecture vaut dix tutoriels.

A toi

Fais `git init` dans `mes-tests-git` (ou `mon-carnet`). Cree un fichier texte. Lance `git status` et lis le resultat a voix haute. Ecris en une phrase : "ce que Git sait de mon dossier maintenant". Garde cette phrase. Au chapitre suivant, elle changera apres ton premier commit.

Chapitre 4 - add et commit (la photo)



add prepare. commit photo. Une etape claire.

Le duo du quotidien : `git add` pour preparer, puis `git commit` pour prendre la photo. Sans ce duo, tu modifies des fichiers, mais Git n'enregistre pas d'etape claire. Avec ce duo, tu construis un album lisible. Chez DanielCraft, on repete : un **commit** = une etape claire. Pas "update". Pas "wip final vrai". Un **message** qui dit quoi / pourquoi. Si demain tu ouvres `log --oneline` et que tu rougis, le probleme n'est pas Git. C'est la legende de la photo.

Lea commit souvent, petit. Une correction de CSS, un commit. Une section de page, un commit. Max commit moins souvent au debut, puis comprend qu'il perd la finesse de l'historique : impossible de revenir juste avant le bug sans tout perdre. Sam interdit les messages "asdf" en atelier. Trois styles, une meme exigence : demain, ton `log` doit se lire comme un journal de bord, pas comme un griffonnage de fin de soiree.

Tu vas aussi rencontrer `status` en boucle. Avant `add`, apres `add`, apres `commit`. Ce n'est pas de la paranoia. C'est le tableau de bord. Tu regardes ce qui est stage, ce qui ne l'est pas, ce qui est ignore. Tu decides. Puis tu photographies. Le geste exact : tu ranges sur la table ce que tu veux dans la photo (`add`), tu declenches (`commit`) avec une legende (`-m`), et `log --oneline` te montre l'album compact.

★ A RETENIR

add prepare. commit photographie. Sans message clair, tu as une photo sans legende.

Ce que ce n'est pas

`add`, ce n'est pas "envoyer sur GitHub". `commit`, non plus. Ce n'est pas non plus une sauvegarde cloud. C'est une photo locale dans l'historique. Ce n'est pas obligatoire d'ajouter tout avec `git add .` les yeux fermes : regarde `status` avant. Tu ne veux pas photographier un secret. Et ce n'est surtout pas `commit --amend` en mode reflexe sur un commit deja partage : prudence. Amend est un outil local, pour corriger la derniere photo avant de la montrer a tout le monde.

Ce n'est pas non plus "un commit par jour maximum". Au contraire : plusieurs petits commits clairs battent un mega-commit de trois jours. Lea le sait. Max l'a appris apres avoir du demeler un commit "toute la feature". Sam chronometre la lisibilite du log, pas le nombre de commits.

Ajouter et committer

```
git add readme.txt
git status
```

Le fichier passe en "staged" (index). Pret pour le commit.

```
git add .
```

Le point = "tout ce qui a change ici". Pratique. Mais regarde `status` avant.

```
git commit -m "Premier commit : readme de base"
```

`-m` = message en une ligne. Prefere un verbe a l'imperatif : "Ajouter", "Clarifier", "Corriger". On detaille les messages au chapitre 12. Des maintenant, vise quelque chose que tu comprendras demain matin.

Voir que ca a marche, puis continuer

```
git status
git log --oneline
```

`status` devrait dire que rien ne reste a commit. `log` montre ta photo. Change `readme.txt`, puis :

```
git status
git add readme.txt
git commit -m "Clarifier le titre du carnet"
```

Chaque commit = une etape claire. Lea lit ses messages comme un journal de bord. Max cree `todo.txt` avec 3 taches, commit, coche une tache, commit encore : son `log --oneline` raconte une histoire en quatre lignes. Sam demande aux eleves de lire le log d'un camarade : si personne ne comprend, on reecrit les prochaines legendes.

Amend (aperçu, avec prudence)

Tu viens de commit et tu as oublie un fichier ?

```
git add fichier_oublie.txt
git commit --amend --no-edit
```

Ca refait le dernier commit. Ne l'utilise pas sur un commit deja pousse si tu travailles en equipe (sauf si tu sais pourquoi). Sam le montre, puis dit "maintenant oublie-le une semaine". Lea amend seulement quand elle est seule sur la branche et que rien n'est parti sur le remote. Max a amend trop tot une fois, force-push ensuite, et a compris pourquoi Sam avait dit non.

Petite histoire

Lea a fait deux commits sur un readme. Le premier "creation". Le second "ajout section contact". Un client a demande "c'etait quoi avant ?". Elle a montre `log` et `diff`. Credibilite immediate. Chez DanielCraft, un historique clair est une forme de professionnalisme - meme en solo. Max, lui, a d'abord commit "aaa" puis "bbb". Quand Sam a projete le log, la classe a ri. Max aussi. Puis il a recommence avec des messages propres. La honte productive marche mieux qu'un long sermon.

Erreur classique

Message "update". `add .` sans lire `status`. Croire que commit pousse en ligne. Oublier `add` puis s'etonner que le commit "n'a rien". Autre piege : un seul mega-commit de trois jours de travail - impossible a relire, impossible a revenir proprement. Encore un piege : committer des fichiers generes ou des secrets "pour plus tard on nettoiera". Non. Plus tard, le secret est deja dans l'historique.

⚠ ATTENTION

`git add .` ajoute tout le dossier. Regarde `status` avant : un `.env` oublie peut finir dans l'historique.

En vrai

Apres chaque commit, `git log --oneline`. Tu dois voir tes messages s'empiler. Si un message te fait honte, c'est un signal : ecris mieux le prochain, pas un roman sur le passe. Fais deux ou trois photos aujourd'hui sur ton carnet de tests. Sens le rythme. C'est ce rythme que tu rejoueras sur un vrai projet.

A toi

Fais 2 commits : d'abord la creation du readme, puis l'ajout d'une ligne. Puis cree `todo.txt` avec 3 taches, commit, coche une tache, commit encore. Quatre photos. Un album qui parle. Demain, relis `log --oneline` sans ouvrir les fichiers : tu dois comprendre l'histoire.

Chapitre 5 - Lire l'historique



Une ligne de commits : A, B, C, D.

L'intérêt de Git, c'est de pouvoir regarder en arrière sans paniquer. Un album sans lecture, c'est une boîte fermée. Avec `log`, `diff` et `show`, tu ouvres la boîte. Tu vois qui, quand, quoi. Tu compares avant de commit. Tu comprends `HEAD`. Tu jettes une modif locale avec prudence. Chez DanielCraft, "lire l'historique" est une compétence égale à "écrire un commit". Sans lecture, tu photographies à l'aveugle. Avec lecture, tu pilotes.

Lea lit `diff` avant chaque commit important. Cinq secondes qui évitent une mauvaise photo. Max a découvert `log --oneline` et a sourit : "ça ressemble à une todo". Sam fait lire un `log` à voix haute en classe : si personne ne comprend les messages, le problème n'est pas Git, c'est la légende des photos. Toi, tu vas apprendre à ouvrir l'album avant de le modifier.

`HEAD` = où tu es maintenant. `HEAD~1` = le commit d'avant. `log` = la liste des photos. `diff` = le détail entre deux états. `show` = le zoom sur une photo. `restore` = "annule mes griffonnages non photographiés sur ce fichier" - attention, tu perds ces griffonnages. Avant de restaurer, demande-toi si tu as vraiment besoin de jeter. Si tu hésites, copie le fichier ailleurs.

★ A RETENIR

`git diff` = changements non stages. `git diff --staged` = ce qui attend le commit. Deux commandes, deux questions différentes.

Ce que ce n'est pas

Lire l'historique, ce n'est pas réécrire le passé (rebase interactif, etc.). Ce n'est pas non plus "revenir en arrière" à chaque commande : `log` regarde, `restore` peut jeter des mods non commit. Ce n'est pas obligatoire de tout comprendre des options graphiques avancées. Et ce n'est surtout pas une perte de temps : cinq secondes de `diff` évitent un mauvais commit.

Ce n'est pas "Git qui modifie quelque chose" quand tu lances `log`. `log` lit. Il n'écrit pas. Si tu as peur de "casser" en lisant, respire. Lire est toujours sûr. Écrire (reset, restore, amend) demande plus d'attention. On sépare les deux familles dans la tête.

♦ ASTUCE

Avant chaque commit important, lance `git diff`. Lea le fait depuis des mois : cinq secondes qui évitent une mauvaise photo.

log et diff

```
git log
```

Tu vois hash, auteur, date, message. Quitte avec `q` si c'est une vue longue.

```
git log --oneline
```

Une ligne par commit. Idéal au début. C'est la vue que tu vas ouvrir le plus souvent.

```
git log --oneline -- readme.txt
```

Voir un fichier à travers le temps. Utile quand tu te demandes "qui a cassé ce CSS ?" ou "quand cette phrase a changé ?".

Avant de commit :

```
git diff
```

Ca montre les modifications non staged. Les `+` et les `-` deviennent vite familiers.

Entre staging et dernier commit :

```
git diff --staged
```

Entre deux commits :

```
git diff HEAD~1 HEAD
```

```
git show HEAD
```

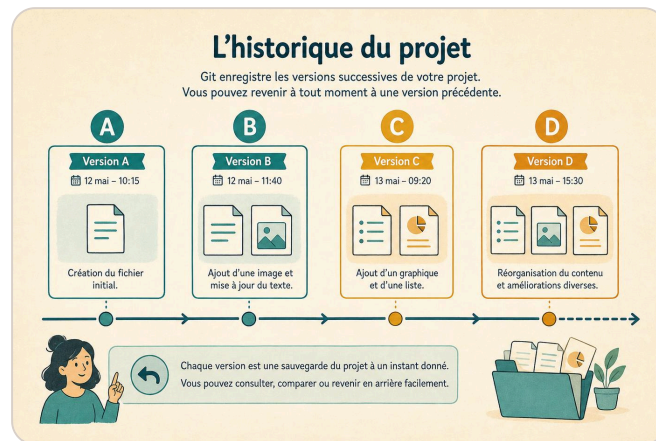
Détail du dernier commit. Hash, message, diff inclus. Sam adore `show` pour expliquer "voilà exactement ce que cette photo contient".

Checkout / restore d'un fichier (aperçu)

Pour jeter les mods non commit d'un fichier :

```
git restore readme.txt
```

(Anciennement `git checkout -- readme.txt`.) Attention : tu perds les changements non sauves dans un commit. Lea le dit a voix haute avant d'appuyer sur Entree. Max copie parfois le fichier ailleurs s'il doute. Sam interdit `restore` en atelier tant que l'eleve n'a pas dit "je jette volontairement". Bon filtre. Chez DanielCraft, on separe clairement "lire" (`log`, `diff`, `show`) et "jeter" (`restore`) : le premier est toujours sur, le second demande une phrase orale avant Entree.



Exemple : des versions du projet qui s'enchainent.

Petite histoire

Max a fait un petit changement, lance `git diff`, vu le `+` et le `-`, compris mieux qu'avec un discours. Sam a demande trois commits avec des messages tres clairs, puis a fait lire le `log --oneline` a un camarade : "est-ce comprehensible ?". Lea a ouvert `git log --oneline --graph --all` une fois et a vu la forme de l'histoire. Chez DanielCraft, on veut que tu voies la forme, pas seulement les commandes. La forme rassure. La forme prepare le chapitre suivant.

Erreur classique

Confondre `diff` et `diff --staged`. Croire que `log` "modifie" quelque chose. Utiliser `restore` comme une baguette sans lire. Autre piege : messages de commit flous qui rendent `log` inutile - tu as l'outil, mais pas l'info. Encore un piege : lancer `diff` apres un `add` et s'etonner que "rien n'apparait" - regarde `diff --staged`. Deux tiroirs. Deux commandes.

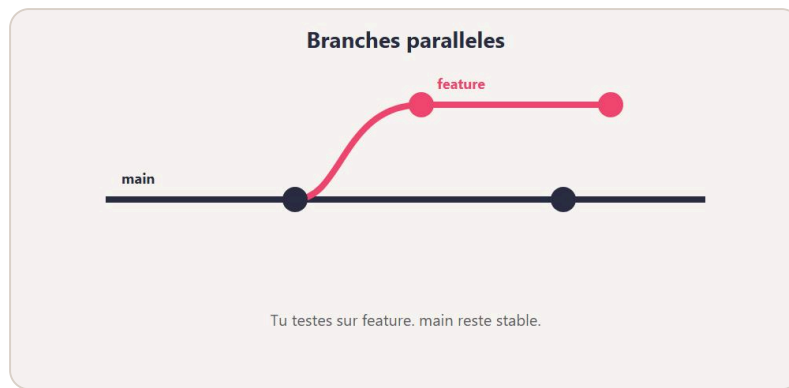
En vrai

Ouvre `git log --oneline --graph --all` une fois. Meme sur un petit projet, tu vois la forme de l'histoire. Puis fais un petit changement, `git diff`, `add`, `commit`, `log --oneline`. Sens la boucle complete : ecrire, voir, preparer, photographier, relire. Si tu hesites entre `diff` et `diff --staged`, pose la question a voix haute : "ai-je deja fait `add` ?" Oui -> staged. Non -> working tree. Chez DanielCraft, cette boucle lit-ecrit vaut mieux qu'une heure de video sans ouvrir le terminal.

A toi

Cree 3 commits avec des messages tres clairs. Demande a quelqu'un (ou a toi demain matin) de lire ton `log --oneline` : est-ce comprehensible ? Si non, reecris ta facon de titrer les prochaines photos. Note aussi la difference entre `diff` et `diff --staged` en une phrase dans `notes.md`.

Chapitre 6 - Les branches



Une branche = une piste parallele pour tester sans casser.

Une **branche**, c'est une piste parallele. Tu experimentes sans casser la version stable. Sur **main**, tu gardes ce qui marche. Sur **idee-couleurs**, tu oses. Si ca marche, tu fusionneras. Si ca ne marche pas, tu jettes la branche. **main** reste intact. Chez DanielCraft, ce reflexe est pro meme en solo : avant chaque experience, nouvelle branche. Ce n'est pas de la paperasse. C'est une assurance.

Lea cree une branche par feature client. Une idee = une piste. Max a decouvert le "retour sur main" et a cru a la magie la premiere fois : les fichiers "revenaient" a l'etat d'avant. Sam fait creer **essai**, ajouter un fichier, revenir : le fichier disparaissait de la vue de travail. Lecon concrete en trente secondes. Toi, tu vas vivre ce moment. Ensuite, les branches ne seront plus abstraites. Elles seront un geste.

Sans branches, tu as un album. Avec branches, tu as des albums paralleles. Tu peux corriger un bug urgent sur **main** pendant qu'une feature avance ailleurs. Tu peux tester une typo risquee sans polluer la version stable. Tu peux jeter une piste sans pleurer. **main** est la route principale. Tu ouvres une route de chantier. Tu y poses des commits. Quand tu reviens sur **main**, tu revois le paysage d'avant. Les commits de la branche n'ont pas disparu : ils sont sur l'autre piste.

★ A RETENIR

main garde ce qui marche. Une branche sert a oser. Si ca rate, tu jettes la piste, pas le projet.

Ce que ce n'est pas

Une branche, ce n'est pas une copie complete lourde du projet "comme un dossier zip". Git est malin : il partage l'historique commun et ne duplique pas tout betement. Ce n'est pas non plus un conflit automatique. Ce n'est pas obligatoire d'avoir dix branches ouvertes. Et ce n'est surtout pas "trop avance pour un debutant" : c'est le coeur de Git.

Ce n'est pas non plus "creer une branche = sauvegarder automatiquement". Tant que tu n'as pas commit sur la branche, tes fichiers modifies restent des griffonnages locaux. Une branche pointe vers un commit. Sans commit, tu n'as pas encore d'histoire sur cette piste. Lea le rappelle souvent. Max l'a appris en creant **essai** puis en oubliant de committer.

Creer, changer, travailler

git branch

Souvent tu as `main` (ou `master` sur de vieux depots).

```
git branch idee-couleurs
git switch idee-couleurs
```

Ou en une commande :

```
git switch -c idee-couleurs
```

(`git checkout -b ...` existe encore. `switch` est plus clair.)

Modifie un fichier. Commit.

```
git log --oneline
```

Ces commits sont sur `idee-couleurs`. `main` n'a pas encore ces changements.

```
git switch main
```

Tes fichiers "reviennent" a l'etat de `main`. Magique la premiere fois. Normal ensuite. Sam fait creer `essai`, ajouter un fichier, commit, revenir sur `main` : le fichier disparaît de la vue. Les eleves comprennent les branches en trente secondes.

Lea nomme ses branches comme des intentions : `fix-header`, `essai-typo`, `feature/contact`. Un nom flou (`test2`) devient une dette mentale en deux semaines. Sam interdit `test`, `tmp`, `new` en atelier. Trop vague. Trop vite oublie.

A quoi ca sert vraiment ?

Tu lances une feature en cours. Tu corriges en urgence sur `main` pendant que tu experimentes ailleurs. Ou tu testes un essai risque, puis tu jettes la branche si ca ne marche pas. Lea a sauve un vendredi comme ca. Max a arrete de "commenter tout le CSS" pour experimenter : il branche. Sam dit aux eleves : "si tu as peur de casser, tu n'as pas encore cree de branche". Phrase un peu dure. Phrase juste.

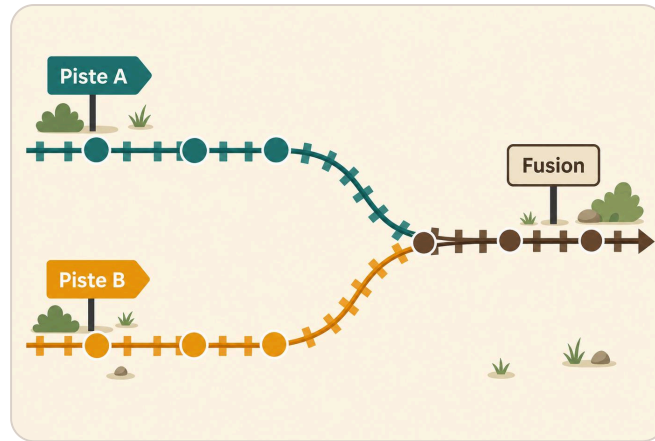
Renommer / supprimer

```
git branch -m ancien-nom nouveau-nom
git branch -d idee-couleurs
```

`-d` refuse s'il reste des commits non fusionnes. `-D` force (prudent). Chez DanielCraft, on prefere comprendre pourquoi `-d` refuse avant de forcer. Souvent, c'est Git qui te protege d'une perte. Ecoute-le. Puis decide.

♦ ASTUCE

Nomme tes branches comme des intentions : `fix-header`, `essai-typo`. Un nom flou (`test2`) devient une dette mentale en deux semaines.



Exemple : deux pistes, puis une fusion.

Petite histoire

Sam a demandé deux branches : `page-a` et `page-b`, un commit différent sur chacune. Les élèves ont noté ce que `log` montrait sur chaque. La différence est devenue visuelle. Lea nomme ses branches clairement. Max a jeté une branche `essai-couleurs` après un échec : `main` intact, zero stress. Chez DanielCraft, c'est le moment où Git cesse d'être "un truc de développeurs" et devient une assurance pour tout projet un peu sérieux.

Erreur classique

Travailler sur `main` "parce que c'est plus simple", puis tout mélanger. Oublier sur quelle branche tu es (`git branch` / `git status` te le disent). Supprimer avec `-D` trop vite. Autre piège : créer une branche sans commit dessus et s'étonner que "rien n'existe". Encore un piège : dix branches ouvertes sans fusionner pendant un mois. Merge souvent. Petits pas. On y vient au chapitre suivant.

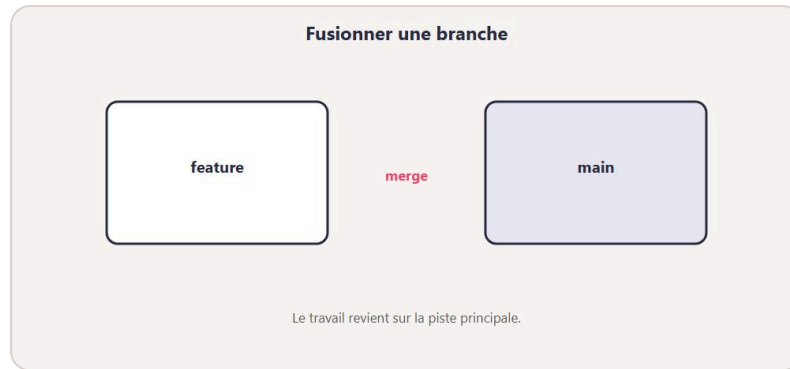
En vrai

Avant chaque expérience : nouvelle branche. Réflexe pro, même en solo. Fais l'exercice `essai` maintenant, pas "plus tard". Sens le retour sur `main`. Ce basculement doit devenir naturel. Ouvre aussi `git log --oneline --graph --all` pour voir plusieurs pistes.

A toi

Crée une branche `essai`. Ajoute un fichier `essai.txt` et commit. Reviens sur `main`. Vérifie que `essai.txt` n'est plus là (normal). Bonus : deux branches `page-a` et `page-b`, un commit différent sur chacune, note ce que `log` montre. Écris en une phrase : "à quoi me sert une branche sur mon prochain projet".

Chapitre 7 - Fusionner (merge)



Ramener le travail d'une branche dans main.

Tu as travaillé sur une branche. Tu veux ramener le travail dans `main`. Le **merge**, c'est le point de jonction. Git essaie de combiner les historiques. Parfois c'est simple (**fast-forward**). Parfois ça crée un commit de fusion. Parfois ça demande ton avis (conflit, chapitre suivant). Chez DanielCraft, on apprend **merge** avant **rebase** : un outil clair d'abord, les subtilités ensuite. Tu n'as pas besoin d'être "clever". Tu as besoin d'être fiable.

Lea fusionne localement pour apprendre, puis passe par des pull requests sur les vrais projets. Max dessine `main` et `feature` qui se rejoignent sur un papier. Sam fait merger deux branches avec chacune un fichier différent : les deux fichiers doivent être présents sur `main`. Preuve concrète. Toi, tu vas merger pour de vrai dans ton carnet de tests. Ensuite, le mot "fusion" ne fera plus peur. Ce sera un geste.

Merge souvent. Petits pas. Une branche qui dort trois semaines accumule les surprises au moment de fusionner. Deux routes : tu ramènes le chantier dans la route principale. Si la principale n'a pas bougé, Git peut juste avancer le pointeur : fast-forward. Si les deux ont avancé, Git crée souvent un **commit de jonction**. Ensuite tu regardes `log --graph`. Puis tu peux supprimer la branche locale fusionnée.

★ A RETENIR

Merge souvent, petits pas. Une branche qui dort trois semaines accumule les surprises au moment de fusionner.

Ce que ce n'est pas

Merger, ce n'est pas "écraser main sans regarder". Ce n'est pas non plus obligatoire de passer par GitHub pour apprendre : en local / solo, **merge** suffit. Ce n'est pas un rebase. Et ce n'est surtout pas une opération magique qui comprend ton intention métier : Git combine des historiques de fichiers. Toi, tu vérifies le résultat. Tu ouvres le site. Tu lances le script. Tu lis **status**. Puis seulement tu souris.

Ce n'est pas non plus "supprimer la branche avant de vérifier". Tu merges. Tu regardes. Tu testes. Puis tu peux `branch -d`. L'ordre compte. Max a supprimé trop vite une fois. Il a dû recréer. Lea vérifie toujours le rendu avant de nettoyer.

♦ ASTUCE

Dessine `main` et `feature` sur un papier avant ton premier merge. Max a fait ça : le graphe Git n'a plus paru magique apres.

ScENARIO de base

```
git switch main  
git merge idee-couleurs
```

Si tout va bien, Git cree souvent un "merge commit" (ou avance juste le pointeur). Tu vois le resultat dans les fichiers + dans `log`.

Parfois Git ouvre un editeur. Tu peux aussi :

```
git merge idee-couleurs -m "Fusion idee couleurs"
```

Le message de merge peut rester simple. L'important : etre sur `main` (ou sur la branche cible) avant de lancer la commande. `status` te le rappelle si tu regardes. Souvent tu es sur `main`, et tu merges la feature dedans. Pas l'inverse, sauf intention claire.

Apres le merge

```
git log --oneline --graph  
git branch -d idee-couleurs
```

Tu peux supprimer la branche locale une fois fusionnee. Lea le fait apres verification visuelle du site. Max regarde d'abord que les fichiers attendus sont la. Sam fait lire le graphe a voix haute : "main a avance, la feature a rejoint". Quand les eleves le disent, c'est compris.

Quand ne pas merger directement ?

Sur GitHub, on passe souvent par une pull request (chapitre 16) : relecture, discussion, CI. En local / solo, `merge` suffit pour apprendre. Les deux cohabitent dans une vraie vie pro. Lea merge en local pour apprendre, PR pour livrer. Max a commence en local uniquement. Sam montre les deux sans dogme. Chez DanielCraft, on veut que tu saches joindre des historiques avant de formaliser la collab.

Merge vs rebase (aperçu)

`merge` : garde l'historique tel quel, ajoute souvent un commit de jonction. `rebase` : rejoue tes commits "par-dessus" une autre branche. Historique plus lineaire, mais plus piegeux. Pour ce livre : maitrise `merge` d'abord. Le rebase, tu le croiseras plus tard (avec prudence, surtout sur branches partagees). Chez DanielCraft, on refuse le rebase "parce que c'est cool" avant de savoir reparer.

Petite histoire

Lea a fusionné `ajout-note` dans `main`, vu le graphe, sourit. Sam a fait merger deux fichiers différents : aucune collision, deux ajouts heureux. Max a voulu "tout rebaser" après une vidéo YouTube : Sam a dit non, pas encore. Une semaine plus tard, Max était content d'avoir un historique compréhensible plutôt qu'un exploit fragile. Chez DanielCraft, on célèbre le merge propre plus que le tour de force.

Erreur classique

Merger sans être sur `main` (ou sur la branche cible). Supprimer la branche avant de vérifier. Confondre merge et push. Autre piège : attendre trois semaines avant de fusionner - plus de divergences, plus de conflits. Merge souvent. Petits pas. Encore un piège : croire que merge "envoie sur GitHub". Non. Merge joint des historiques locaux. Push envoie ensuite, si tu as un remote.

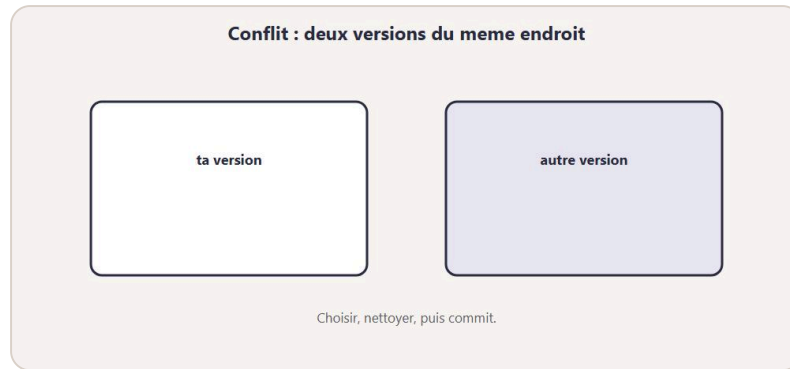
En vrai

Fais un dessin : `main` et `feature` qui se rejoignent. Le merge, c'est le point de jonction. Puis fais l'exercice pour de vrai dans ton carnet de tests. Regarde `log --graph --oneline`. Sens la forme. C'est cette forme que tu reverras toute ta vie de projet versionné.

A toi

Crée une branche `ajout-note`. Commit un fichier. Merge dans `main`. Regarde `log --graph --oneline`. Bonus : deux branches avec chacune un fichier différent, merge les deux dans `main`, vérifie que les deux fichiers sont présents. Note en une phrase ce qui s'est passé : fast-forward ou commit de jonction.

Chapitre 8 - Les conflits



Deux versions du meme endroit : choisir, puis commit.

Un **conflit**, c'est Git qui ne sait pas choisir. Deux versions du meme endroit. Il te demande d'arbitrer. Ce n'est pas un echec. C'est Git qui dit : "decide, humain". Chez DanielCraft, on provoque un conflit volontairement dans les ateliers, pour enlever la peur. Lea les resolvait deja. Max paniquait. Sam a transforme la panique en methode. Toi, tu vas faire pareil. La premiere fois, les **marqueurs** <<<<<<< font peur. La dixieme, tu souffles, tu choisis, tu enlèves les marqueurs, tu **add**, tu **commit**. Muscle. Pas magie.

Un conflit arrive quand deux histoires touchent la meme zone du meme fichier. Toi hier et toi aujourd'hui. Toi et Lea. Deux branches. Local et remote. Peu importe. Git ne "plante" pas. Il refuse de mentir en choisissant a ta place. Tu peux reduire la frequence avec de petits commits et des merges frequents. Tu ne peux pas promettre zero conflit a vie. Mieux vaut savoir nager que croire que la mer n'existe pas.

Deux stylos ont ecrit sur la meme ligne. Git pose les deux versions dans le fichier avec des balises. Tu gardes A, ou B, ou un melange intelligent. Tu enlèves les balises. Tu dis a Git : "voila la version choisie" (**add** + **commit**). Si tu es perdu au milieu, **git merge --abort** te ramene avant le merge. Apprends ce frein d'urgence tot. Max l'appelle "le bouton panique honnete". Sam l'enseigne avant meme de provoquer le conflit.

⚠ ATTENTION

Ne committe jamais un fichier qui contient encore <<<<<<<. Git croit que c'est resolu. Ton programme, lui, plantera.

Ce que ce n'est pas

Un conflit, ce n'est pas "Git est casse". Ce n'est pas non plus "ton projet est perdu". Ce n'est pas automatique d'accepter "incoming" sans lire. Et ce n'est surtout pas une raison de jeter Git. C'est le prix normal de deux personnes (ou deux branches) qui touchent la meme zone. Ce n'est pas non plus une excuse pour laisser les marqueurs dans le code "on verra demain". Demain, le build casse. Aujourd'hui, tu nettoies.

Ce n'est pas non plus "toujours accepter les deux versions". Parfois le melange intelligent est juste. Parfois une seule version est juste. Parfois il faut ecrire une troisieme version a la main. Lis. Comprends. Puis choisis. Lea refuse les boutons "accepter tout" tant qu'elle n'a pas lu le diff.

Comment ca arrive et a quoi ca ressemble

Sur `main`, tu modifies la ligne 1 de `readme.txt`. Sur une branche, tu modifies aussi la ligne 1. Tu merges : boum, conflit.

Dans le fichier :

```
<<<<<< HEAD
version de main
=====
version de la branche
>>>>>> idee-couleurs
```

Tu gardes ce que tu veux. Tu enlèves les marqueurs `<<<<<<` etc. Complètement. Aucun marqueur ne doit rester. Ensuite seulement tu stages et tu conclus.

Resolving

Ouvre le fichier. Corrige a la main. Fais `git add fichier_corrige.txt`, puis `git commit` pour conclure le merge. Ou, si ton outil propose "accepter actuel / entrant / les deux", OK - mais comprends ce que tu acceptes. Lea refuse les boutons automatiques tant qu'elle n'a pas lu le diff. Max a accepte "les deux" une fois sans lire : le fichier compilait pour Git, pas pour le navigateur. Sam fait relancer le site apres chaque resolution. Le conflit "resolu" doit aussi etre juste pour le produit.

♦ ASTUCE

Lea refuse les boutons "accepter tout" tant qu'elle n'a pas lu le diff. Dix secondes de lecture valent une heure de debug.

Annuler un merge en cours

Si tu es perdu :

```
git merge --abort
```

Retour avant le merge. Respire. Recommence plus tard, plus calme. Ce n'est pas un aveu d'echec. C'est de la gestion de risque. Lea abort quand elle est fatiguee. Max abort quand le fichier devient illisible. Sam celebre l'abort autant que la resolution propre. Les deux sont des competences.

Eviter les conflits (un peu)

Fais des petits commits. Parle-toi en equipe. Merge souvent (pas attendre 3 semaines). Evite que deux personnes editent la meme zone sans coordination. Decoupe les fichiers si possible. Chez DanielCraft, on prefere la prevention polie a l'heroisme du dimanche soir. Lea se repartit les fichiers avec un collegue. Max annonce "je touche au header". Sam impose des zones differentes en binome au debut. Moins de drames. Plus d'apprentissage utile.

Petite histoire

Max a provoqué un conflit volontairement, vu les marqueurs, choisi la version claire, termine le merge. Il a dit : "c'est juste un formulaire à remplir". Sam a noté 3 règles perso au tableau pour limiter les conflits. Lea a raconté un conflit client résolu en dix minutes parce que les commits étaient petits. La culture compte autant que la commande. Chez DanielCraft, on veut que tu sortes de ce chapitre avec moins de peur, pas avec zéro conflit à vie - promesse impossible.

Erreur classique

Laisser les marqueurs <<<<<< dans le fichier et committer. Accepter "les deux" sans relire. Abandonner le projet au lieu d'abort. Autre piège : résoudre "au feeling" sans lancer le programme / ouvrir la page - un conflit "résolu" peut être logique pour Git et faux pour le produit. Encore un piège : paniquer et lancer dix commandes d'affilée. Une action. Un status. Ensuite seulement.

En vrai

Un conflit n'est pas un échec. C'est Git qui dit : "décide humain". Provoque-en un volontairement aujourd'hui, tant que le dossier est un terrain de jeu. Sens les marqueurs. Enlève-les. Termine. Ensuite tu sauras.

A toi

Provoque un conflit volontairement sur `readme.txt`. Résols-le. Termine le merge. Puis note 3 règles perso pour limiter les conflits dans ton prochain projet. Garde-les à côté de ton `.gitignore` mental. Relis-les avant ta première collab réelle.

Chapitre 9 - .gitignore



Ignorer secrets et dossiers lourds des le debut.

Certains fichiers ne doivent jamais aller dans Git. Secrets, caches, gros binaires, dossiers de dependances, fichiers d'OS... Un `.gitignore` a la racine dit a Git : "ne propose meme pas d'ajouter ca". Ce n'est pas une liste magique universelle. C'est une **politique** claire pour ce projet. Chez DanielCraft, on le pose tot, avant le premier push public - surtout avant d'apprendre `push`. Un secret committe reste dans l'historique. Meme si tu le "supprimes" apres. Meme si tu rougis.

Lea a deja pousse un `.env` par accident. Elle a change le secret ensuite. Lecon chere. Max ignore `Thumbs.db` et ses PDF de brouillon. Sam fait creer un faux `.env` et verifier qu'il n'apparait plus dans `status`. Exercice simple, impact immense. Toi, tu vas ecrire ton premier `.gitignore` aujourd'hui, meme sur un carnet de tests. Habitude posee avant le danger reel.

Git propose des fichiers a photographier. `.gitignore` met des post-it "ne pas photographier" sur certaines etiquettes : `.env`, `node_modules/`, `__pycache__`, `.venv/`. Tu adaptes la liste. Tu verifies avec `status`. Si un secret a deja ete photographie, tu l'enlèves de l'album (`rm --cached`) sans forcement le supprimer de ton bureau. Puis tu tournes la page avec un commit. Si la photo est deja sur GitHub, tu changes la serrure (le secret), pas seulement l'album.

⚠ ATTENTION

`.gitignore` n'efface pas un fichier deja suivi. Si `.env` est deja dans l'historique, il faut `git rm --cached` + commit. Et changer le secret si c'etait un vrai.

Ce que ce n'est pas

`.gitignore`, ce n'est pas un antivirus. Ce n'est pas non plus "effacer le fichier de ton disque". Ce n'est pas retroactif si le fichier est deja suivi : il faut `git rm --cached` puis commit, et si le secret est deja en ligne, changer le secret. Et ce n'est surtout pas optionnel sur un projet qui touche des cle API, des mots de passe, des dumps. Ce n'est pas non plus "ignorer tout avec `*`" : tu ne versionnerais plus ton code. Sois precis.

Ce n'est pas non plus un fichier qu'on ecrit "plus tard, apres le push". Plus tard, le mal est fait. Ecris-le tot. Lea le cree souvent juste apres `init`. Max le cree avant le premier `add .`. Sam l'exige avant toute demo publique.

★ A RETENIR

Avant le premier push public : relis `git status` . Cherche mots de passe, cles, dumps. Puis seulement, pousse.

Creer le fichier

A la racine du projet, cree `.gitignore` :

```
# secrets
.env
*.key

# python
__pycache__/
.venv/

# node
node_modules/

# OS / editeur
.DS_Store
Thumbs.db
.idea/
.vscode/
```

Adapte a ton projet. Tu n'as pas besoin de tout. Tu as besoin de ce qui te concerne. Lea part d'un template GitHub, puis coupe ce qu'elle ne comprend pas. Mieux qu'une liste copiee aveugle. Max ajoute `*.pdf` sur ses carnets, sauf un manuel utile. Sam fait relire chaque ligne a voix haute en classe : "pourquoi ignore-t-on ca ?"

Verifier et reparer

```
git status
```

Les fichiers ignores ne doivent plus apparaitre comme "a ajouter".

Si tu as deja `add` un fichier sensible :

```
git rm --cached fichier.env
```

Ca l'enleve de l'index, pas de ton disque. Puis commit. Et ajoute-le au `.gitignore` . Si le secret est deja sur GitHub : change le secret (mot de passe, cle). L'enlever de l'historique est un autre niveau. Pour ce livre, retiens : prevention d'abord, `rm --cached` ensuite, rotation du secret si fuite. Lea a appris ca a ses depens. Max a eu de la chance. Sam raconte l'histoire pour que personne n'ait a la vivre.

Patterns utiles et templates

```
*.log  
tmp/  
build/  
dist/  
*.pdf  
!docs/manuel.pdf
```

! peut re-inclure une exception. Commence simple. Complexifie seulement si besoin. GitHub propose des `.gitignore` tout faits (Python, Node...). Tu peux t'en inspirer. Chez DanielCraft, on veut que tu comprennes chaque ligne que tu gardes. Une ligne incomprise est une ligne dangereuse ou inutile.

Petite histoire

Max a écrit un `.gitignore` pour un mini site HTML/CSS/JS avec `dist/` ignore. Il a build, vu que `status` restait propre. Sam a fait relire `status` avant tout push : "cherche mots de passe, clés, dumps". Lea garde une checklist de poche. Chez DanielCraft, le professionnalisme commence souvent par ce que tu n'enregistres pas. Lea a aussi montré à un client un dépôt propre : pas de secrets, pas de `node_modules`, README clair. Le client n'a pas vu Git. Il a vu de la rigueur.

Erreur classique

Ignorer après avoir commit le secret, sans `rm --cached`. Croire que `.gitignore` efface un fichier déjà suivi. Committer `.env.example` utile... et aussi `.env` réel. Autre piège : ignorer trop large (`*` partout) et ne plus versionner ton code. Sois précis. Encore un piège : croire que "privé sur GitHub" égal "secret safe". Privé aide. Ça ne remplace pas `.gitignore` ni la rotation d'une clé fuitee.

En vrai

Avant le premier push public : relis `git status`. Cherche mots de passe, clés, dumps, photos perso. Puis seulement, pousse. Fais-le même sur un faux projet. Le rituel compte plus que le risque immédiat. Chez DanielCraft, on préfère dix secondes de relecture à une heure de nettoyage d'historique. Si tu hésites sur une ligne du `.gitignore`, demande-toi : "est-ce du code source utile demain ?" Oui -> versionne. Non -> ignore.

A toi

Ajoute un `.gitignore` à ton carnet de tests. Crée un faux `.env` avec `SECRET=demo`. Vérifie que Git l'ignore. Bonus : `.gitignore` pour un mini site avec `dist/` à ignorer. Écris en trois lignes ta checklist perso "avant push".

Chapitre 10 - GitHub, c'est quoi ?



Git = outil. GitHub = depot en ligne.

GitHub heberge tes depots Git sur internet. Sauvegarde distante, collaboration, issues, pull requests, vitrine README. Ce n'est pas Git lui-meme. C'est un hotel pour tes albums Git, avec des couloirs pour travailler a plusieurs. Chez DanielCraft, on separe bien les deux mots : tu peux maitriser Git sans GitHub, mais en 2026 tu croiseras GitHub (ou un cousin) tres vite. GitLab, Bitbucket, Forgejo : meme famille d'idee. On apprend avec GitHub parce que c'est partout. Les gestes se transferent.

Lea pousse ses projets clients en prive. Max met son carnet en prive aussi, puis invite Sam en lecture. Sam montre la page GitHub au tableau : fichiers, commits, README. La "preuve visuelle" rassure autant que la commande. Toi, tu vas creer un compte, un depot de test, lier **origin**, pousser. Quand tu rafraichiras la page et verras tes fichiers, tu sentiras le soulagement. C'est normal. C'est le but.

Ton ordi a l'album local. GitHub a une copie distante (**origin**). Tu lies les deux avec **remote add** . Tu envoies avec **push** . Tu recuperes avec **pull** (chapitre suivant). Le **README.md** est la vitrine : cinq lignes claires battent un roman vide. HTTPS ou SSH : deux portes d'entree. Choisis-en une et avance. Tu pourras changer plus tard. Chez DanielCraft, on veut que tu pusses, pas que tu te perdres dans un debat de porte.

★ A RETENIR

Creer le depot GitHub sans cocher "Add a README" si tu as deja un historique local. Ca evite les premiers commits divergents au debut.

Ce que ce n'est pas

GitHub, ce n'est pas obligatoire pour versionner en local. Ce n'est pas non plus le seul hebergeur. Ce n'est pas "le cloud magique qui commit pour toi". Et ce n'est surtout pas un endroit pour coller des secrets. Avant le premier push public ou meme prive partage, relis **status** et ton **.gitignore** . Ce n'est pas non plus "finir le projet" : un README de cinq lignes claires bat un roman vide.

Ce n'est pas non plus "public par default pour tout". Prive existe. Utilise-le pour les projets clients, les notes perso, les exercices incomplets. Lea pousse en prive par default. Max aussi. Sam montre public seulement quand le contenu est volontairement partage.

Compte et nouveau depot

Cree un compte sur github.com. Confirme l'email. Optionnel mais recommande : active la double authentification. Deux minutes qui evitent des sueurs froides.

Sur GitHub : New repository. Choisis un nom, par exemple `mon-carnet` . Public ou prive. Tu peux ne pas cocher "Add a README" si tu as deja un depot local - evite les histoires de premiers commits divergents au debut. Si GitHub cree un README et que toi aussi tu as un historique, tu auras deux debuts a fusionner. Faisable. Embettant pour un premier push. Evite-le.

Lier ton depot local

Sur la page du depot vide, GitHub montre des commandes. En general :

```
git remote add origin https://github.com/TON_COMPTE/mon-carnet.git
git branch -M main
git push -u origin main
```

`origin` = surnom classique de ton remote principal.

```
git remote -v
```

Verifie que l'URL est la bonne. Max a pousse une fois vers le mauvais depot parce qu'il avait copie trop vite. `remote -v` avant le premier push, c'est pas cher.

SSH ou HTTPS ?

HTTPS est simple au debut (parfois un token). SSH utilise des cles et devient tres confortable au quotidien. Les deux marchent. Lea estensee en SSH apres deux semaines. Max est reste en HTTPS avec token. Sam montre les deux sans dogme. Si l'auth rate, lis le message. Token expire ? Cle mal ajoutee ? Droits manquants ? Une cause. Une correction. Pas dix essais au hasard.

♦ ASTUCE

Lea pousse en prive par default. Max active la double authentification des qu'il cree son compte. Deux minutes qui evitent des sueurs froides.

README.md

Sur GitHub, `README.md` s'affiche en vitrine. Ecris 5 lignes : but du projet + comment lancer. Puis commit et push. Verifie le rendu. C'est ton premier "produit visible". Lea soigne le README autant que le code sur les petits projets. Max a compris le jour ou un ami a compris son carnet sans l'appeler. Sam refuse de "considerer le projet fini" tant que la page GitHub ne montre rien de comprehensible.

Petite histoire

Lea a pousse, rafraichi la page, vu ses fichiers, souffle. Max a ajoute un vrai README clair le soir meme. Sam a refuse de valider un atelier tant que l'URL ne montrait pas un titre et deux phrases utiles. Trois standards, une meme idee : le distant existe pour etre lu. Chez DanielCraft, la page GitHub fait partie du livrable, pas du bonus.

Erreur classique

Créer un README sur GitHub et un historique local divergent sans savoir merger. Pousser sans `.gitignore`. Oublier la double authentification sur un compte pro. Autre piège : déposer un projet perso public avec des emails clients dedans. Prive existe. Utilise-le. Encore un piège : croire que "c'est sur GitHub" égal "c'est sauvegarde pour toujours" sans jamais vérifier le push. Rafraichis. Regarde. Confirme.

En vrai

Après le push, rafraichis la page GitHub. Tu dois voir tes fichiers. Soulagement. Puis améliore le README. Commit. Push. Vérifie le rendu. Sens la boucle locale -> distante. C'est cette boucle que tu rejoueras chaque jour utile.

A toi

Crée un dépôt GitHub de test (privé OK). Branche `origin`. Fais `push` de ta branche `main`. Ajoute un vrai `README.md` clair. Commit. Push. Vérifie. Note l'URL quelque part : tu en auras besoin pour le mini-projet.

Chapitre 11 - clone, push, pull



clone / pull / push : le dialogue avec le distant.

Voici le dialogue avec le serveur. **clone** recupere un depot existant. **push** envoie tes commits locaux. **pull** ramene les commits distants chez toi. Sans ce dialogue, Git reste un album solo. Avec ce dialogue, tu rentres dans le jeu equipe / sauvegarde distante. Chez DanielCraft, le reflexe du matin en collab, c'est **git pull** avant de recoder. Pas glamour. Fiable.

Lea pull, code, commit, push. Max oubliait le pull, se prenait des **rejected**, apprenait. Sam fait cloner un petit depot public en lecture seule pour explorer un **log** vivant. Trois gestes, une meme conversation. Toi, tu vas sentir la boucle complete : modifier, photographier, envoyer, recuperer. Quand quelque chose bloque, tu liras le message jusqu'au bout. Souvent, la reponse est dedans.

```
toi (local) --push--> GitHub
toi (local) <--pull-- GitHub
```

fetch telecharge les infos sans fusionner tout de suite. **pull** $\approx$ **fetch** + merge (en simplifiant). Au debut, **pull** suffit. Ahead = tu as des commits locaux non pousses. Behind = le remote a avance. Rejected = tu as essaye de pousser alors que le remote avait des commits que tu n'avais pas. Ces trois mots reviennent tout le temps. Apprends-les comme un vocabulaire de survie. Chez DanielCraft, on les ecrit au tableau la premiere semaine : trois mots, trois reflexes.

★ A RETENIR

Push refuse (**rejected**) ? Le remote a avance. **pull** d'abord, resous si besoin, puis **push**. Lis le message jusqu'au bout.

Ce que ce n'est pas

push, ce n'est pas **commit**. Si tu n'as pas commit, tu n'as rien a envoyer. **pull**, ce n'est pas "ecrire ton travail local sans regarder" - mais oui, ca peut creer des merges / conflits si les deux cotes ont avance. **clone**, ce n'est pas **init** : clone copie + configure souvent **origin**. Et ce n'est surtout pas une excuse pour pousser sur **main** partage sans convention d'equipe. Ce n'est pas non plus **force push** "parce que ca bloque" - hors sujet ici, et dangereux en equipe.

Ce n'est pas non plus "clone met a jour un dossier existant". Non : clone cree une copie. Pour mettre a jour, `pull`. Max a clone dix fois le meme depot dans dix dossiers avant de comprendre. Sam le montre explicitement. Lea rit, puis confirme : un dossier, un remote, des pulls. Chez DanielCraft, on prefere un `pull` du matin a dix clones paniques.

clone, push, pull

```
git clone https://github.com/TON_COMPTE/mon-carnet.git
cd mon-carnet
```

`clone` = copie + lien `origin` deja configure.

```
git push
```

La premiere fois (si pas de `-u`) :

```
git push -u origin main
```

```
git pull
```

Reflexe avant de recommencer a coder le matin : `git pull`.

```
git fetch
```

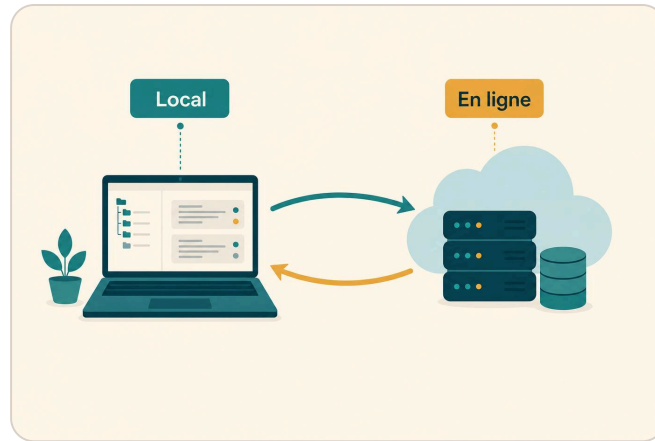
Apercu utile. Pas obligatoire chaque jour pour ce livre. Utile quand tu veux regarder avant de fusionner. Lea fetche sur les gros projets. Max pull sur les petits. Les deux sont honnetes selon le contexte.

Erreurs frequentes

`rejected` : le remote a des commits que tu n'as pas -> `pull` d'abord. Auth ratee : mauvais token / session. Mauvaise branche : tu pushes `main` alors que tu es ailleurs. Chez DanielCraft, on lit le message d'erreur jusqu'au bout avant de retaper la meme commande dix fois. Lea souligne la derniere ligne du message. Max a commence a le faire apres une heure perdue. Sam exige la traduction orale avant toute aide.

⚠ ATTENTION

`push` n'envoie que ce qui est commit. Si tu as modifie sans committer, rien ne part. Max a appris ca en voyant "Everything up-to-date" alors qu'il avait du travail non sauve.



Exemple : local et en ligne qui se parlent.

Petite histoire

Max a modifié un fichier, commit, push. Puis il a édité une ligne sur GitHub via l'interface. En local, `git pull` a ramené le changement. La boucle est devenue concrète. Lea, sur un projet à deux, a imposé "pull avant push". Les conflits ont baissé. Sam a fait cloner un dépôt open source, lire `log --oneline`, ne rien pousser. Explorer sans casser : excellent sport. Chez DanielCraft, on aime autant savoir lire un dépôt étranger que savoir pousser le sien.

Erreur classique

Pousser sans pull sur une branche partagée. Croire que clone "met à jour" un dossier existant. Confondre le dossier parent et le dossier clone. Autre piège : force push "parce que ça bloque". Encore un piège : coder toute la journée sans pull le matin, puis découvrir un mur de conflits à 18h. Pull tot. Petits pas. Moins de drames.

En vrai

Sur un projet à deux : toujours `pull` avant de pousser. Ça diminue les conflits. Sur un projet solo : push quand même assez souvent pour avoir une copie distante. Une fois par jour utile, c'est déjà bien. Lea pousse à chaque feature terminée. Max pousse avant de fermer le PC. Sam pousse avant la pause déjeuner en demo. Choisis un rituel. Tiens-toi y. Si tu vois `rejected`, ne retape pas `push` dix fois : lis, `pull`, résous, puis `push`. Chez DanielCraft, un remote à jour bat une sauvegarde USB oubliée dans un tiroir.

A toi

Modifie un fichier, commit, puis push. Change quelque chose sur GitHub via l'interface (petit edit). Fais `git pull` en local. Bonus : clone un petit dépôt public open source (lecture seule), explore `log --oneline`, ne push rien. Note en une phrase la différence entre ahead et behind.

Chapitre 12 - De bons messages de commit



Un message = une phrase d'action claire.

Un bon **message** sauve ton futur toi. Et ton equipe. Et le client qui te demande "pourquoi ce total a change en mars". L'**historique** n'est pas un tas de "update" empiles. C'est un journal de bord : chaque commit raconte une intention. Si tu ne comprends plus un message d'hier, le probleme n'est pas Git. C'est la legende de la photo. Chez DanielCraft, on traite le message comme une competence pro, pas comme une corvee qu'on reportera au prochain sprint.

Lea ecrit a l'imperatif court : "Ajouter", "Corriger", "Clarifier". Elle ajoute le pourquoi quand ce n'est pas evident : "Corriger le total TTC (oubli de la TVA)". Max a du desapprendre "final final 2" et "aaa" apres avoir ri de son propre **log**. Sam projette un **git log --oneline** propre et un sale devant la classe : la difference se voit en dix secondes. Toi, tu vas viser le propre. Pas le parfait academique. Le lisible. Le jour ou tu debug a 23h un vendredi, tu remercieras ton toi du matin.

Imagine que tu lis ton historique a voix haute dans six mois, sans ouvrir les fichiers. La premiere ligne doit tenir en une respiration (50 a 72 caracteres ideaux). Prefere un verbe a l'**imperatif** : "Ajouter", pas "J'ai ajoute". Explique le pourquoi si ce n'est pas evident. Un commit = une intention coherente. Pas "j'ai change 40 fichiers sans rapport parce que c'etait vendredi". Pas non plus 40 micro-commits illegibles ("fix", "fix2"). Le juste milieu vient avec l'experience. En attendant, vise "une idee claire par photo".

★ A RETENIR

Un commit = une intention. La premiere ligne resume. Le pourquoi reste si ce n'est pas evident.

Ce que ce n'est pas

Un bon message, ce n'est pas un roman de vingt lignes a chaque fois. Ce n'est pas non plus un emoji seul ou un numero de ticket copie-colle sans contexte. Ce n'est pas obligatoire d'adopter Conventional Commits des le jour un - utile en equipe, apercu ici - mais ce n'est pas une religion. Et ce n'est surtout pas "je corrigerai plus tard" : tu ne reecris pas toute l'historique tous les matins. Ecris bien maintenant.

Ce n'est pas non plus "un label **feat:** devant asdf". Un prefixe ne sauve pas un message vide. L'intention d'abord. Le format ensuite. Lea l'utilise sur les projets clients a plusieurs. Max s'en passe en solo, mais garde des messages clairs sans prefixe. Les deux vont.

Oui et Non (exemples concrets)

Les bons messages ressemblent a ca : "Corriger le total TTC (oubli de la TVA)", "Ajouter la page contact", "Retirer le fichier .env du suivi". Tu comprends l'action et souvent la raison. Les mauvais ressemblent a ca : "update", "fix", "wip", "asdf", "final final 2". Ils ne disent rien. Ils polluent l'album. Quand Sam voit "fix stuff" dans un depot eleve, il demande : "fix quoi, pour qui, pourquoi ?". Si tu ne peux pas repondre, le message est mauvais. Reecris-le avant de pousser, ou amend si tu n'as pas encore pousse et que tu es seul sur la branche.

Lea compare deux projets : l'un avec des messages nets, l'autre avec des "update". Sur le premier, elle retrouve un bug en deux minutes. Sur le second, elle lit des diffs pendant une heure. Max a compris le jour ou il a du expliquer un changement a un client : le message clair lui a evite de rougir au telephone.

Corps optionnel et Conventional Commits

Parfois le titre ne suffit pas. Git accepte un corps de message :

```
git commit -m "Corriger le calcul du panier" -m "Le total ignorait les codes promo. Desormais appliques avant TVA."
```

Tu verras parfois en equipe des prefixes comme `feat:`, `fix:`, `docs:` :

```
feat: ajouter export PDF
fix: corriger lien casse du sommaire
docs: preciser l'install Windows
```

Utile quand toute l'equipe parle la meme langue. Pas obligatoire pour apprendre. L'important : l'intention lisible, pas le label a la mode. Chez DanielCraft, on juge un historique a voix haute : si Sam comprend sans contexte, c'est bon.

Exemple de fil propre

Voici un historique qu'on peut lire en dix secondes :

```
Initialiser le projet
Ajouter la page d'accueil
Corriger le lien du menu mobile
Ignorer le fichier .env
Documenter l'installation dans le README
```

Tu vois l'histoire du projet sans ouvrir un seul fichier. C'est ca, le but. Si tu rougis en lisant ton propre log, change ta facon d'ecrire demain, pas dans six mois. La honte productive marche. Max l'a vecue. Lea aussi, plus tot dans sa carriere. Sam la provoque volontairement en classe avec bienveillance.

♦ ASTUCE

Projette ton `git log --oneline` a voix haute. Si Sam comprend sans contexte, c'est bon. Si tu rougis, change ta facon d'ecrire demain.

Petite histoire

Lea a du retrouver pourquoi un total etait faux sur un site e-commerce. Le message "Corriger le total TTC (oubli de la TVA)" l'a menee en trente secondes au bon commit. Sur un autre projet legacy, "update" l'a forcee a lire des diffs pendant une heure. Max a reecrit sa facon d'ecrire apres avoir projete son log devant un ami developpeur. Sam fait ecrire trois messages "pro" avant le premier vrai commit en classe. Rituel simple, resultat durable. Chez DanielCraft, le message clair est une forme de respect pour l'equipe future - et pour toi-meme.

Erreur classique

Messages vides de sens : "update", "fix", "changes". Tout mettre dans un seul commit "grosse feature" sans decoupage. Mentir dans le message ("fix typo" alors que tu as refactorise toute l'archi). Copier le style Conventional sans en respecter l'esprit : un label `feat:` devant "asdf" reste un mauvais message. Autre piege : ecrire pour une IA, pas pour un humain qui debug a 23h un vendredi. L'humain d'abord. Toujours.

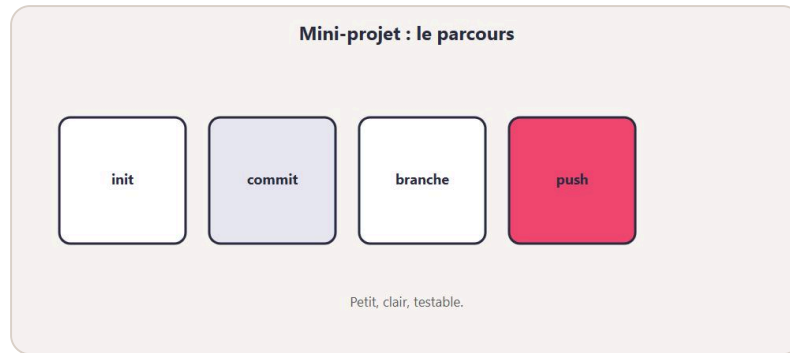
En vrai

Lis ton `git log --oneline` sur ton carnet de tests. Si tu ne comprends plus un message d'hier, c'est le signal. Renomme ta facon d'ecrire des maintenant. Fais 3 commits sur ton carnet avec des messages "pro". Compare avant/apres. Tu sentiras la difference tout de suite.

A toi

Reecris ces messages pourris en bons messages : `update`, `fix stuff`, et `aaa`. Puis applique le meme standard sur ton prochain vrai commit. Garde l'exemple de fil propre sous les yeux une semaine. Note ce qui t'a le plus aide : l'imperatif, le pourquoi entre parentheses, ou la taille d'un commit.

Chapitre 13 - Mini-projet : carnet versionne



init, commit, branche, push.

On enchaîne le vrai **workflow** du quotidien. Pas un exercice isolé dans le vide : un petit projet **carnet-git** avec README, notes, historique propre, branche de feature, ignore, push GitHub. Chez DanielCraft, ce mini-projet sert de preuve. Quand tu l'as fait une fois sans paniquer, tu peux rejouer le film complet sur un vrai site, un script, ou un dossier de formation. Le livre cesse d'être théorique. Il devient muscle.

Lea le fait en quinze minutes maintenant qu'elle a l'habitude. Max a mis quarante minutes la première fois, vingt la deuxième. Sam chronomètre la classe et célèbre le process, pas la vitesse. Toi, vise la clarté d'abord. La vitesse viendra quand tu n'auras plus à relire chaque chapitre. Si tu bloques sur une étape, c'est un signal : rouvrir le chapitre concerne dix minutes, puis reviens.

Local d'abord. Histoire propre ensuite. Branche pour une idée isolée. Merge quand c'est prêt. Ignore les secrets avant qu'ils ne passent. Remote pour la copie lointaine. Preuve URL dans le README pour montrer que tu sais boucler la boucle. Tu reconnaitras cette séquence dans presque tous les projets sérieux : init, commit, branch, merge, ignore, push. Une fois dans les doigts, tu la rejoues partout. Chez DanielCraft, on forme au transfert, pas à la recopie.

♦ ASTUCE

Chronomètre-toi la première fois, puis refais le mini-projet une semaine plus tard. Sam a vu le temps chuter de moitié chez ses élèves - la qualité montait en même temps.

Ce que ce n'est pas

Ce n'est pas un monorepo d'entreprise avec vingt équipes. Ce n'est pas non plus "juste push n'importe quoi pour cocher la case". Ce n'est pas obligatoire d'ouvrir une pull request dès la première passe - c'est la variante avancée, utile si tu as déjà lu le chapitre 16. Les critères minimum restent simples : au moins 4 commits lisibles, une branche fusionnée, un **.gitignore**, un projet visible sur GitHub (privé OK). Sans ça, le mini-projet ne compte pas comme réussi.

Ce n'est pas non plus une course. Si tu prends une heure la première fois en comprenant chaque geste, c'est mieux qu'un sprint opaque de douze minutes. Lea préfère la clarté. Max aussi, maintenant. Sam note le process dans la grille d'évaluation, pas le chrono seul.

But

Un petit projet `carnet-git` avec un `README.md`, un `notes.md`, un historique propre, une branche de feature, et un push GitHub. C'est ton portfolio Git minimal. Lea le montre parfois a un client pour prouver qu'elle sait travailler proprement. Max l'a montre a un ami developpeur. Sam le demande en fin de module comme preuve.

Etapes

1. Local

```
mkdir carnet-git
cd carnet-git
git init
```

Ecris un README (but + auteur). Commit : `Initialiser le carnet`

2. Notes

Ajoute `notes.md` avec trois idees courtes. Commit : `Ajouter les premieres notes`

3. Branche

```
git switch -c feature/note-git
```

Ajoute une note sur Git. Commit. Reviens sur `main` et merge.

4. Ignore

Ajoute `.gitignore` avec `.env` et un faux secret local. Commit : `Ignorer les fichiers sensibles`

5. GitHub

Cree le depot distant sur GitHub. Puis `remote add` + `push -u origin main`

6. Preuve

Copie l'URL GitHub dans ton `README.md`. Commit + push.

A chaque etape, lance `status`. Lis. Avance. Si quelque chose cloche, stop. Une commande. Un controle. Chez DanielCraft, c'est la methode anti-panic du mini-projet.

Variante avancee

Ouvre une issue "Ameliorer le README". Cree une branche depuis cette idee. Ouvre une pull request (chapitre 16). Merge via la PR au lieu d'un merge local silencieux. Lea le fait systematiquement sur les projets clients, meme en solo, pour s'obliger a resumer son travail avant de fusionner. Max l'a ajotee la deuxieme fois. Sam la propose en bonus, jamais en piege.

⚠ ATTENTION

Criteres minimum : 4 commits lisibles, une branche fusionnee, un `.gitignore`, un depot visible sur GitHub (prive OK). Sans ca, le mini-projet ne compte pas.

Petite histoire

Max a invité un ami en lecture seule sur le depot. L'ami a compris le README sans appeler Max. Max a été fier - et a corrigé deux messages de commit flous juste après, parce que l'historique était aussi visible que le README. Sam a fait refaire le mini-projet une semaine plus tard en classe : le temps a chuté, la qualité a monté, les erreurs (oublier `.gitignore`, messages "update") ont presque disparu. Chez DanielCraft, on aime cette deuxième passe. La première apprend. La seconde ancre. Lea le refait parfois quand elle change d'ordinateur : rituel de remise en main.

Erreur classique

Sauter `.gitignore` "parce que c'est un faux projet". Messages "update" ou "test". Merger sans vérifier `status` avant et après. Pousser un secret "pour voir si Git le détecte" - spoiler : Git ne juge pas, il enregistre. Vouloir la variante PR avant d'avoir un historique local propre. Une marche après l'autre. Autre piège : oublier l'URL dans le README - tu as poussé, mais tu n'as pas fermé la boucle "preuve".

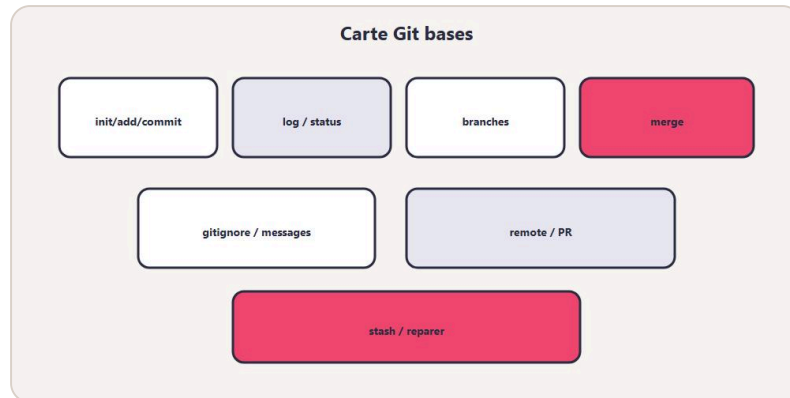
En vrai

Chronomètre-toi sans stress. Vise moins de 20 minutes une fois à l'aise. Puis invite quelqu'un en lecture seule si tu peux : le regard extérieur révèle les trous du README et les messages flous. Si tu es seul, relis ton `log --oneline` à voix haute comme si tu expliquais à Lea. Si ça sonne clair, c'est bon.

A toi

Livrable : URL du dépôt (privé OK) + 4 commits lisibles minimum + branche fusionnée + `.gitignore`. Écris aussi en trois lignes ce qui t'a freiné pendant le mini-projet. On réutilise ça dans les ateliers. Garde l'URL : tu y reviendras au quiz et au bravo.

Chapitre 14 - Ce qu'il faut retenir (bases)



Carte des bases Git.

Si tu ne devais garder qu'une page avant stash, undo et pull requests, ce serait celle-ci. Tu as parcouru la **boucle quotidienne**, les commandes vitales, les **zones** de Git, les habitudes qui tiennent, et les erreurs classiques que tout le monde fait au moins une fois. Chez DanielCraft, on veut une carte mentale actionnable, pas un glossaire de cinquante termes à reciter. Lis cette page comme un plan de métro : tu sais où tu es, tu sais quelle ligne prendre. Si tu bloques plus loin, tu reviens ici dix minutes.

Lea résume en une phrase : "status, add, commit, push ; pull souvent si collab". Max résume : "photos locales, copie distante, branches pour oser". Sam résume : "lis status, ne commit aucun secret, explique tes commits". Trois phrases, un même livre. Si tu peux dire la tienne sans notes, tu es prêt pour la suite. Sinon, ce n'est pas un échec. C'est un signal de pratique. Rouvre les chapitres 4, 6 et 11. Cinq minutes chacun. Puis réessaie.

★ A RETENIR

Status d'abord, secret jamais, branche pour oser. Cette phrase résume mieux le livre qu'une liste de vingt commandes.

Ce que ce n'est pas

Ce recap, ce n'est pas "tu as fini Git". Ce n'est pas non plus une liste à mémoriser comme un poème. Ce n'est pas un examen piège. C'est une carte. Tu t'orientes. Tu avances. Tu reviens si besoin. Ce n'est surtout pas une excuse pour sauter les ateliers : la carte ne remplace pas le terrain. Lea relit cette page avant un nouveau projet client. Max la recite le lundi matin. Sam la fait écrire sans notes en fin de module.

Boucle quotidienne

Le geste qui revient chaque jour ressemble à ça : tu ouvres le projet, tu fais **git status** pour voir où tu en es, tu prépares avec **add**, tu photographies avec **commit**, tu envoies avec **push** si tu travailles avec un remote. Si tu collabores, tu **pull** souvent pour ne pas diverger trop longtemps. Ce n'est pas glamour. C'est fiable. Lea le fait avant le café. Max le fait avant de fermer la journée. Sam le fait en démo devant la classe. Chez DanielCraft, l'habitude bat le talent du soir où tu "feras gaffe".

Commandes vitales

Tu démarres avec `git init` (nouveau projet) ou `git clone` (copie d'un depot existant). Tu regardes l'état avec `git status` - ton tableau de bord. Tu prépares avec `git add`, puis tu photographies avec `git commit`. Pour l'histoire : `git log --oneline`. Pour les pistes parallèles : `git switch` et `git branch`, puis `git merge` quand tu es prêt à fusionner. Avec le distant : `git pull` pour recevoir, `git push` pour envoyer, et `git remote -v` pour vérifier que tu parles au bon depot. Tu n'as pas besoin de tout savoir par cœur le jour un. Tu as besoin de savoir ou chercher quand un message te parle. L'aide `git help` existe. Les chapitres aussi.

Zones

Git organise ton travail en zones. Tu modifies des fichiers (working directory). Tu prépares ce que tu veux photographier avec `add` (index / staging). Tu figes avec `commit` (historique local). Tu copies loin avec `push` (remote). Si tu confonds deux zones, `status` te le dit en clair : untracked, staged, modified. Apprends à le lire comme un tableau de bord, pas comme une insulte. Sam dit à ses élèves : "status avant, status après". Ça évite la moitié des paniques. Lea le fait sans y penser. Max a mis une semaine à en faire un réflexe. Toi aussi, tu y arriveras.

Habitudes

Fais `status` avant et après chaque action importante. Ecris des messages clairs, pas "update". Mets un `.gitignore` tot, dès le premier commit si possible. Préfère les petites branches pour une idée, une feature, un fix. Ne commit jamais un secret : `.env`, clés API, mots de passe. Merge souvent pour éviter les grosses divergences. Pull avant de pousser en équipe. Chez DanielCraft, ces habitudes battent les "astuces avancées" regardées sur YouTube trop tot.

♦ ASTUCE

Imprime mentalement l'aide-memoire express. Lea l'a collée sur un post-it une semaine. Ensuite, c'était dans les doigts.

Erreurs classiques (recap)

On commit parfois sur la mauvaise branche - ça se repare si tu sais lire status. Un push peut être refusé si le remote est en avance : pull d'abord, réparer, repush. Un `conflit` laisse des marqueurs `<<<<<<` si tu oublies de nettoyer avant le commit de merge. Et `git add .` sans regarder, c'est le classique des secrets qui passent ou des fichiers temporaires qui polluent l'historique. Si tu les reconnais au premier regard, tu as gagné. Max les a tous faits une fois. Lea aussi. Sam les montre volontiers. Personne n'est "nul en Git". Certains n'ont pas encore assez pratiqué.

Aide-memoire express

Pour l'état : `git status`. Pour préparer : `git add`. Pour la photo : `git commit -m "..."`. Pour l'histoire : `git log --oneline`. Pour une branche : `git switch -c nom`. Pour fusionner : `git merge nom`. Pour envoyer : `git push`. Pour recevoir : `git pull`. Imprime cette page mentalement. Max la recite une fois le lundi matin. Lea n'en a plus besoin. Sam la redistribue chaque promo.

Petite histoire

Lea a colle l'aide-memoire sur un post-it une semaine entiere. Ensuite, c'etait dans les doigts et le post-it est parti a la poubelle. Max a rate un push "rejected", a lu le chapitre pull, a corrige en dix minutes au lieu de paniquer une heure. Sam a fait ecrire sans notes la boucle add/commit/push et le role d'une branche en fin de module. Ceux qui y arrivaient passaient a stash et PR sans drame. Ceux qui bloquaient rouvraient les chapitres 4, 6 et 11 dix minutes, puis reessaient. Chez DanielCraft, c'est exactement le contrat : carte claire, pratique honnete, zero humiliation.

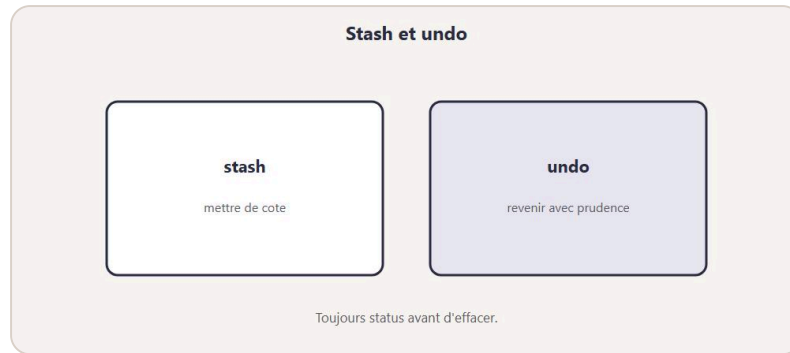
Suite immediate

Deux chapitres un cran au-dessus t'attendent : stash + annuler proprement, puis les pull requests. Puis trois ateliers pour casser et reparer, un quiz, et un bravo. Tu n'es pas "fini". Tu es pret a monter d'un etage sans perdre le sol. C'est mieux que "fini". C'est solide.

A toi

Sans notes : ecris la boucle add/commit/push et a quoi sert une branche. Si ca vient fluide, avance vers stash et PR. Sinon, rouvre les chapitres 4, 6 et 11 dix minutes chacun, puis reessaie. Ce recap ne remplace pas la pratique - il te dit ou repartir sans te disperser.

Chapitre 15 - Stash et annuler (avec prudence)



Mettre de cote ou revenir en arriere, avec prudence.

Parfois tu dois changer de branche alors que tu as des modifs pas pretes. Parfois tu as commit trop vite avec un mauvais message. Parfois tu dois annuler quelque chose deja partage avec l'equipe. Ce chapitre te donne des outils - et surtout une **regle d'or** qui evite les catastrophes. Chez DanielCraft, on enseigne la prudence avant la puissance : un **reset --hard** mal place fait plus de degats qu'un conflit bien resolu en quinze minutes. La puissance sans frein n'est pas du professionnalisme. C'est de la roulette.

Lea **stash** pour repondre a une urgence client sans committer un brouillon a moitie. Max a decouvert **revert** apres avoir panique sur **main** partage. Sam interdit **--hard** en atelier tant que **status** n'a pas ete lu a voix haute devant la classe. Trois postures, une meme discipline : lire, comprendre si c'est local ou partage, puis agir une fois. Toi, tu vas apprendre a choisir. Pas a enchaîner trois resets "au hasard" en esperant que ca marche.

Regle d'or : en local, pas pousse -> **reset** possible avec prudence. Deja pousse / equipe -> **revert** (ou discussion avant tout geste heroique). Avant tout geste destructeur : **git status**, **git log --oneline -5**, puis demande-toi "est-ce pousse ?", "suis-je seul sur la branche ?". Si tu as un doute, copie le dossier ailleurs. Respire. Puis choisis une commande, pas trois d'un coup. Max appelle ca "la pause cafe anti **--hard**". Lea appelle ca "la checklist". Sam appelle ca "la survie".

⚠ ATTENTION

reset --hard sur un commit deja partage peut effacer le travail des autres. En equipe, prefere **revert**. C'est le geste "adulte".

Ce que ce n'est pas

Annuler, ce n'est pas "Git efface internet" ni "personne ne verra jamais". Ce n'est pas non plus gratuit : **restore** jette des modifs non commit. **reset --hard** jette encore plus, sans filet. Ce n'est pas interchangeable au hasard : **stash** met de cote temporairement, **restore** annule local sur un fichier, **reset** reecrit l'historique local, **revert** ajoute un commit d'annulation propre. Et ce n'est surtout pas la meme chose avant ou apres un push partage. La question "est-ce deja pousse ?" change tout. Si tu ne sais pas repondre, tu n'es pas pret a taper **--hard**.

Ce n'est pas non plus "stash = sauvegarde eternelle". Stash est un tiroir temporaire. Ne laisse pas pourrir dix stash oublies pendant trois mois. Lea nettoie sa pile chaque vendredi. Max a perdu le fil d'un stash trop vieux. Sam limite a deux stash max en atelier.

stash

Quand tu es en plein milieu d'un travail et qu'une urgence tombe :

```
git stash
git switch autre-branche
# ... tu fais le fix urgent ...
git switch -
git stash pop
```

`stash` = met de cote proprement. `stash pop` = recupere et enleve de la pile. Pour voir ce qui dort :

```
git stash list
```

Utile. Pas un tiroir magique infini. Si `stash pop` cree un conflit, c'est normal parfois : tu as change la meme zone entre-temps. Resous comme un conflit classique. Puis continue. Lea l'a vecu. Cinq minutes. Pas de drame.

restore et reset (aperçus)

Pour annuler des modifs sur un fichier non commit :

```
git restore fichier.txt
```

Revient a la derniere version commit pour ce fichier. Les changements non commits sont perdus. Pour retirer du staging sans perdre le fichier :

```
git restore --staged fichier.txt
```

Pour annuler un commit local pas encore pousse :

```
git reset --soft HEAD~1
```

Annule le commit, garde les changements stages. Ou :

```
git reset HEAD~1
```

Annule le commit, garde les changements non stages. Enfin, le marteau :

```
git reset --hard HEAD~1
```

Revient en arriere et jette les modifs. Puissant. Dangereux. Pas sur un commit deja partage sans discussion d'equipe. Sam le montre une fois, puis range le marteau. Lea ne l'utilise presque jamais. Max l'a utilise trop tot, une fois, sur un depot de test. Heureusement, pas en prod.

revert (plus sur en equipe)

```
git revert HEAD
```

Cree un nouveau commit qui annule le precedent. L'historique reste honnete : l'erreur est visible, l'annulation aussi. Prefere ca sur `main` partage. Chez DanielCraft, c'est souvent le geste "adulte" quand plusieurs personnes lisent le meme depot. Tu n'effaces pas le passe. Tu ajoutes une correction visible. Les autres comprennent. L'histoire reste lisible. Max a fait un commit "oops", puis `revert`, puis regarde le `log` : l'erreur etait la, l'annulation aussi, personne n'a perdu le fil.

★ A RETENIR

En local, pas pousse : reset possible. Deja pousse / equipe : revert. Stash = tiroir temporaire, pas sauvegarde eternelle.

Petite histoire

Max a fait un commit "oops" sur une branche deja mergee sur `main` partage. Il a panique une seconde, puis `revert`, puis regarde le `log` : l'erreur etait la, l'annulation aussi, l'histoire restait lisible. Lea a stash, change de branche, corrige un bug hot pour un client, revenu sur sa feature, `stash pop` avec un petit conflit qu'elle a resolu en cinq minutes. Sam a fait copier le dossier avant un `--hard` "pour voir" en atelier. La copie n'a pas servi ce jour-la. Le reflexe, si. Bonne assurance. Chez DanielCraft, on celebre le geste juste plus que le geste spectaculaire.

Erreur classique

`--hard` sur un commit pousse "parce que personne ne regardera". `stash pop` avec conflits sans comprendre d'ou ils viennent. `commit --amend` apres push sur une branche partagee. Autre piege : enchaîner trois resets "au hasard" en esperant que ca marche - une action, un controle `status`, ensuite seulement la suivante. Encore un piege : confondre restore (fichier local) et revert (nouveau commit d'annulation). Deux outils. Deux moments. Deux risques.

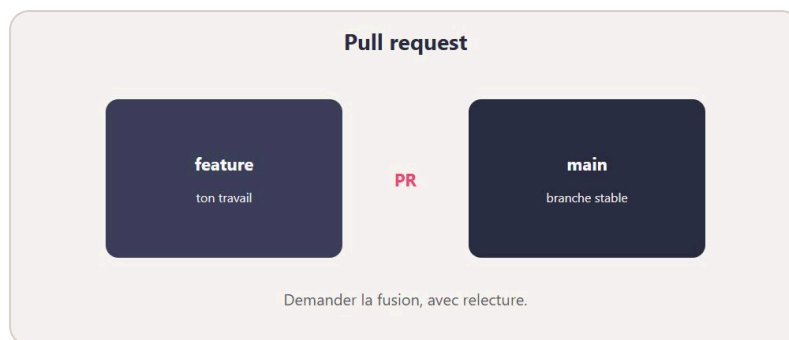
En vrai

Avant un `--hard`, respire. Copie le dossier ailleurs si tu as un doute. Modifie un fichier sur ton carnet de tests, fais `stash`, verifie que `status` est propre, puis `stash pop`. Tu verras que rien n'a disparu dans le neant. Git range. Il ne te punissait pas. Ensuite fais un petit `revert` sur une branche de test. Regarde le log. Sens l'honnetete de l'historique.

A toi

Fais un commit "oops" sur une branche de test. Annule-le avec `revert`. Regarde le `log`. Bonus : scenario stash complet (modifs -> stash -> switch -> retour -> pop). Ecris en trois lignes ta regle d'or perso : quand tu utilises restore, reset, revert, stash. Colle ca dans `notes.md`. Tu y reviendras a l'atelier reparer.

Chapitre 16 - Pull requests (PR)



Une pull request, c'est demander de fusionner ton travail.

Une **pull request**, c'est "peux-tu fusionner mon travail dans la branche principale ?". C'est le coeur de la **collaboration** sur GitHub et la plupart des hébergeurs modernes. Tu proposes. Quelqu'un lit. On discute. On merge. L'historique reste plus clair qu'un push silencieux sur **main** à 23h un vendredi. Chez DanielCraft, même en solo, une PR t'oblige à résumer ton travail avant de fusionner. C'est sain. Ça évite les merges "fix stuff" sans contexte.

Lea ouvre des PR pour chaque feature client, même petite. Max a commencé en mergeant tout seul en local, puis a adopté les PR pour s'entraîner avant de rejoindre une équipe. Sam fait lire deux PR open source en classe : le format revient partout, du petit projet perso au gros framework. Toi, tu vas en ouvrir une, même pour une ligne de README. L'objectif n'est pas la paperasse. C'est la relecture. C'est le pont social au-dessus du pont technique Git.

Tu crées une branche pour une idée. Tu committes proprement. Tu push la branche sur le remote. Sur GitHub, tu ouvres Compare & pull request. Tu écris but, changes, comment tester. Quelqu'un review (ou toi-même le lendemain avec un œil frais). On merge. En local, tu reviens sur **main** et tu **pull** pour récupérer la fusion. Sans description, le pont est glissant. Lea met toujours "comment tester" même quand elle merge seule. Chez DanielCraft, "comment tester" est non négociable sur une PR sérieuse.

★ A RETENIR

Une bonne description PR : but en deux lignes, liste des fichiers changes, comment tester. Sans ça, la review double de temps.

Ce que ce n'est pas

Une PR, ce n'est pas un virus ni une formalité inutile réservée aux grosses boîtes. Ce n'est pas non plus "seulement pour les équipes de dix personnes". Ce n'est pas un commit : c'est une proposition de fusion entre branches, avec discussion. Ce n'est pas obligatoire d'utiliser squash si ton équipe préfère merge commit - demande la convention, note-la. Et ce n'est surtout pas une excuse pour une PR monstrueuse de 40 fichiers sans description : personne ne reviewera correctement. Préfère trois petites PR à une usine à gaz.

Ce n'est pas non plus "obligatoire d'attendre quelqu'un d'autre". En solo, tu peux ouvrir, relire le lendemain, merger toi-même. L'œil frais compte. Lea le fait souvent. Max aussi, maintenant. Sam l'impose comme exercice : review le lendemain, pas dans la seconde.

Scenario type

Tu crees une branche avec `git switch -c feature/login`. Tu fais tes commits locaux avec des messages clairs. Tu push : `git push -u origin feature/login`. Sur GitHub, tu cliques Compare & pull request. Tu remplis titre + description. Quelqu'un review, commente des lignes si besoin. On merge sur GitHub. En local : `git switch main` puis `git pull`. La boucle est complete. Max l'a faite dix fois en solo avant de la faire en equipe : zero surprise le jour J. Lea la rejoue sur chaque feature. Sam la chronometre en atelier sans stresser sur la vitesse.

Pourquoi pas merger en silence ?

Ca permet la relecture, la discussion, les tests automatiques (CI) si tu en ajoutes plus tard, et un historique plus clair avec une trace de la decision. Meme en solo, une PR t'oblige a resumer : "qu'est-ce que j'ai change et pourquoi ?". Draft PR : tu peux ouvrir en brouillon pour montrer l'avancement sans demander le merge tout de suite. Lea adore les drafts sur les gros sujets client : le client voit l'avancement, personne ne merge par accident. Max a decouvert les drafts apres une merge prematuree. Sam les recommande des qu'une feature depasse une demi-journee.

♦ ASTUCE

Lea ouvre des draft PR sur les gros sujets : tu montres l'avancement sans demander le merge tout de suite. Utile pour debloquer une discussion.

Bonne description

Voici un modele simple qui marche :

```
## But
Ajouter un formulaire de contact.

## Changes
- page contact.html
- validation email basique

## Test
Ouvrir /contact et envoyer un faux message.
```

En review, on commente des lignes precises. Tu poussees des commits sur la meme branche : la PR se met a jour automatiquement. Simple et puissant. Chez DanielCraft, on refuse le dogme "squash obligatoire" : on veut une convention ecrite, meme courte, dans CONTRIBUTING.md. Coherent bat clever.

Merge buttons

Tu verras souvent Create a merge commit, Squash and merge (souvent propre pour petites features), et Rebase and merge. Chacun laisse une trace differente dans l'historique. Demande la convention de ton equipe. En solo, choisis-en une et tiens-toi y. L'important : etre coherent, pas etre "le plus clever". Lea squash souvent sur les petites features. Max garde le merge commit pour voir la jonction. Sam montre les trois sans religion.

Petite histoire

Max a ouvert une PR vers `main` sur son carnet-git, merge lui-meme apres relecture le lendemain, puis `pull` en local. La boucle est devenue reelle, pas theorique. Lea a mis un modele de description dans son README (section Contributing) pour ne plus reinventer la roue. Sam a fait comparer une PR claire et une PR "fix stuff" en timing : le temps de review doublait sur la mauvaise. La qualite de la demande change la qualite de la reponse. Toujours. Chez DanielCraft, c'est une lecon de collab autant qu'une lecon de Git.

Erreur classique

PR sans description ("voir commits" n'est pas une description). Travailler directement sur `main` "parce que je suis seul" jusqu'au jour ou tu ne l'es plus. Force push sur la branche de PR partagee sans prevenir. Ouvrir une PR avant d'avoir un `.gitignore` et un README - tu demandes une review sur un chantier sale. Autre piege : PR de 800 lignes "parce que c'etait plus simple". Personne ne reviewera bien. Decoupe.

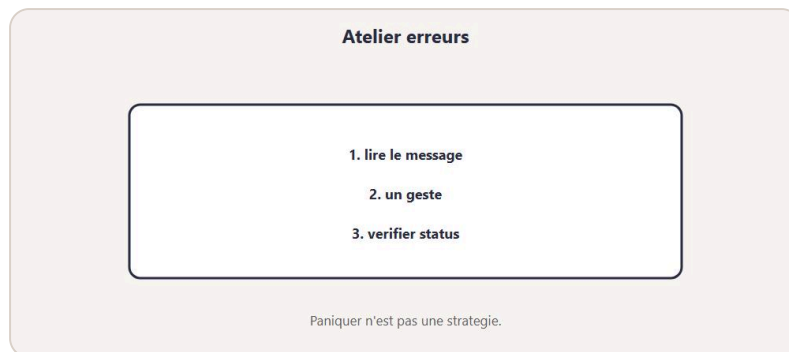
En vrai

Lis 2 PR open source sur GitHub (Documentation, typo fix, petit feature). Tu verras le meme format partout. Puis ouvre la tienne, meme pour une ligne de README. Merge-la proprement. Refais `git pull` en local. C'est ton diplome du chapitre. Garde le modele de description. Tu le reutiliseras.

A toi

Cree une branche avec 1 commit utile. Push. Ouvre une PR vers `main`. Merge-la toi-meme apres relecture. Puis `git pull` en local. Bonus : ecris un modele de description PR dans ton README (section Contributing). Garde-le pour les projets futurs.

Chapitre 17 - Atelier : lire les messages Git



Lire le message, puis corriger un geste a la fois.

Git parle anglais souvent. On traduit. Une **erreur** ou un **status**, c'est un GPS, pas une insulte personnelle. Chez DanielCraft, cet atelier forme l'autonomie : si tu sais lire quatre messages courants sans Google, tu te debloques seul neuf fois sur dix. Lea gagne des heures par mois. Max a arrete de paniquer devant le pave rouge du terminal. Sam exige la derniere partie du message avant toute aide en classe : "traduis d'abord, on discute ensuite". Toi, tu vas te constituer un petit lexique vivant, dans un fichier, avec tes mots.

Objectif : reconnaitre et reparer. Duree : 25 a 40 minutes. Matériel : ton carnet de tests + un remote GitHub si possible (prive OK). Sans remote, les exercices ahead/behind perdent une partie de leur sens - fais-les quand tu peux. Un atelier fait a fond bat trois ateliers survoles en dix minutes. Sam chronometre le process, pas la vitesse de clic.

Le terminal t'envoie un message. Tu lis la fin d'abord - souvent la plus claire. Tu traduis en une phrase francaise. Tu lances **status** pour situer la carte. Tu choisis une action. Tu verifies. Si ca n'a pas marche, tu ne lances pas dix autres commandes. Tu relis. Tu ajustes. C'est un dialogue. Pas un combat. Chez DanielCraft, le rouge veut dire "attention", pas "fin du monde".

★ A RETENIR

Ahead = push. Behind = pull. Rejected = pull puis push. Conflict = editer + add + commit. Garde ces reflexes la premiere semaine.

Ce que ce n'est pas

Ce n'est pas un dictionnaire exhaustif de Git. Ce n'est pas non plus "cliquer jusqu'a ce que ca parte" en esperant le hasard. Lis. Traduis en francais simple. Une action. Verifie avec **status**. Ce n'est pas une course. Ce n'est pas "Googler la premiere commande sans comprendre si c'est pousse ou non". La methode DanielCraft est lente au debut, rapide ensuite. Lea le confirme. Max aussi, apres avoir clique au hasard trop longtemps.

Traductions utiles

Quand Git dit "nothing to commit, working tree clean", ca veut dire : rien a photographier, tout est deja commit. OK, tu peux coder ou changer de branche. "Untracked files" : des fichiers pas encore suivis. **add** si tu les veux, **.gitignore** si tu ne les veux jamais. "Your branch is ahead of origin/main" : tu as des commits locaux non poussees. **git push**. "Your branch is behind" : le remote a avance. **git pull** avant de

continuer ou de pousser. "rejected ... fetch first" : push refuse, le serveur a des commits que tu n'as pas. Tire d'abord, resous, repush. "merge conflict" : ouvre les fichiers marques, nettoie les <<<<<<, add, commit de merge. "Permission denied" ou probleme d'auth : identifiants HTTPS, cle SSH, ou droits sur le depot.

Ces phrases reviennent. Apprends-les comme un vocabulaire de survie. Pas comme une poesie. Lea les a dans un fichier. Max aussi. Sam les projette.

Methode DanielCraft

Lis d'abord la derniere partie du message. C'est souvent la plus claire. Ne panic pas sur le pave rouge : rouge = attention, pas fin du monde. Lance `git status` pour savoir ou tu es exactement. Une action a la fois. Si tu changes dix choses sans relire, tu ne sauras plus ce qui a repare quoi. Lea garde un fichier `atelier-messages-git.md` dans chaque nouveau projet avec ses traductions perso. Chez DanielCraft, on veut ce fichier plus qu'une capture d'ecran oubliee.

⚠ ATTENTION

Ne clique pas au hasard jusqu'a ce que ca parte. Lis, traduit, une action, verifie avec `status`. C'est la methode DanielCraft.

Exercice 1 - Ahead / behind (10 min)

Fais un commit local sans push. Lis le status "ahead". Push. Puis, si tu peux, edite un fichier directement sur GitHub (interface web) et `pull` en local pour voir "behind" disparaitre apres synchro. Note la sequence exacte. Max l'a colle dans son carnet : "ahead = j'ai oublie de push". Ecris la tienne avec tes mots.

Exercice 2 - Rejected (10 min)

Provoque un push reject (pull manquant) et repare. Note la sequence exacte qui t'a debloque. Souvent : pull, resoudre conflit eventuel, push. Sam fait cet exercice en binome : l'un push, l'autre push aussi sans pull - le reject arrive naturellement. Solo, tu peux editer sur GitHub puis push depuis local sans pull : meme lecon.

Exercice 3 - Conflit (10 min)

Provoque un conflit volontaire (meme fichier modifie en local et sur le remote, ou deux branches). Resous-le. Verifie qu'aucun marqueur <<<<<< ne reste avant le commit. Relis le diff une fois : c'est bon entrainement pour la vraie vie. Lea refuse de conclure tant que le rendu n'est pas verifie.

Exercice 4 - Secret (8 min)

Ajoute un faux secret au mauvais moment, puis retire-le avec `rm --cached + .gitignore` + commit propre. Si c'etait un vrai secret deja pousse, tu changerais aussi le secret cote serveur - Git ne "descommit" pas l'historique facilement. Note la difference entre "retire du suivi" et "efface du passe public". Important.

Petite histoire

Max a provoqué un push rejected volontairement un samedi matin. Il a lu le message, tire, repousse. Dix minutes plus tard, il expliquait "ahead" et "behind" à un camarade sans notes. Lea garde son fichier `atelier-messages-git.md` dans chaque nouveau projet client. Sam le consulte en fin de promo : les mêmes erreurs reviennent toujours aux mêmes endroits - push oubliée, pull oubliée, conflit panique. L'atelier ne les élimine pas. Il les rend prévisibles. Chez DanielCraft, prévisible bat héroïque.

Erreur classique

Cliquer au hasard jusqu'à ce que ça parte. Changer dix choses d'un coup sans relire `status`. Googler la première commande sans comprendre si c'est pousser ou non. Faire l'atelier sans remote : ahead/behind restent abstraits. Autre piège : noter les traductions sans refaire les exercices - le cerveau classe ça comme "lu", pas "su". Les mains d'abord. Les notes ensuite.

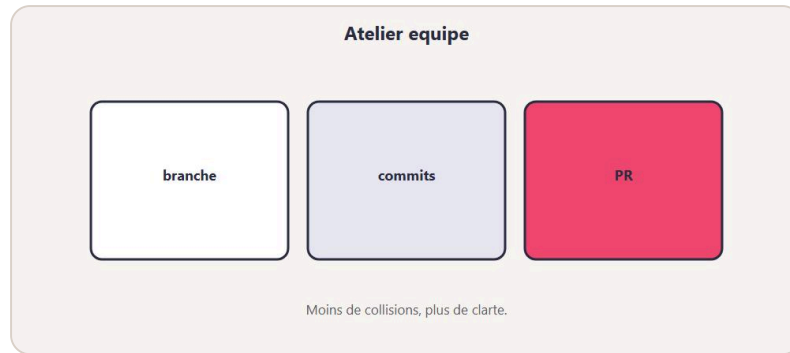
En vrai

La prochaine fois qu'un message apparaît en rouge, écris ta traduction en une phrase avant d'agir. Ce micro-rituel change tout. Lea le fait depuis six mois. Max aussi. Sam l'impose en classe. Toi, commence aujourd'hui. Une phrase. Puis une action. Puis status.

A toi

Fais les quatre exercices. Livrable : un fichier `atelier-messages-git.md` avec 4 messages traduits, une séquence de réparation, 5 lignes de leçons. Puis explique à voix haute, comme à Max ou Lea, la différence entre "ahead" et "behind". Si tu y arrives sans notes, c'est bon signe.

Chapitre 18 - Atelier : workflow d'equipe (simule)



Branches + PR pour moins de collisions.

Meme seul, tu peux simuler deux personnes. L'idee n'est pas de "faire semblant pour le prof". C'est d'entrainer la vraie boucle : branche, push, **PR**, review, merge, pull. Chez DanielCraft, cet atelier prepare mieux a une equipe reelle qu'un chapitre theorique sur "la collab". Quand tu arrives en stage ou en freelance multi-dev, tu as deja joue Alice et Bob. Le stress baisse. Les reflexes montent. Tu n'inventes pas les regles sous pression. Tu les as deja ecrites.

Lea joue Alice et Bob sur deux dossiers clones. Max protege **main** sur GitHub apres s'etre force-push une fois de trop sur un depot de test (heureusement, pas en prod). Sam colle une mini charte dans **CONTRIBUTING.md** et la fait lire a voix haute avant le premier commit. Toi, tu vas jouer le jeu jusqu'au bout : deux PR mergees, historique lisible, charte ecrite. Objectif : 2 PR mergees, historique lisible. Duree : 40 a 60 minutes.

Deux personnes. Un depot. Une **main** protegee. Des branches courtes. Des PR petites. Une review meme rapide. Un merge. Un pull. La culture se code dans **CONTRIBUTING.md** autant que dans les habitudes. Chez DanielCraft, dix lignes de charte prevennent dix conflits evitables. Lea le sait. Max l'a appris. Sam l'enseigne avant le premier binome.

⚠ ATTENTION

`git push --force` reecrit l'histoire distante. Sur une branche partagee, ca peut effacer le travail des autres. A bannir sur **main** tant que tu ne maitrises pas.

Ce que ce n'est pas

Ce n'est pas une invitation a `push --force` sur **main** "parce que ca debloque". Ce n'est pas non plus "commit direct sur main parce que je suis seul donc les regles ne comptent pas". Les regles d'equipe s'entrainent avant d'etre imposees par un lead dev. Ce n'est pas obligatoire d'etre deux humains physiques : deux dossiers clones du meme depot suffisent. Alice a la main le matin, Bob l'apres-midi. Ou deux fenetres, deux navigateurs. Ce n'est pas non plus "provoquer un conflit des la premiere minute" - garde ca pour l'atelier conflits. Ici, on apprend a collaborer proprement d'abord.

Idee et tour de jeu

Deux dossiers locaux : `alice/carnet` et `bob/carnet`. Tous deux clones du meme depot GitHub (ton carnet-git ou un depot de test). Alice cree une branche `feature/alice`, commit sur une zone de fichier (ex : README), push, ouvre une PR. Bob (toi, plus tard) fait `pull` sur `main` apres le merge. Bob cree `feature/bob` sur une autre zone (ex : notes.md), push, PR. Alice review - meme si c'est toi sous un autre compte ou le lendemain avec un oeil neuf. Regle d'or : une zone de fichier differente par personne au debut. Ca limite les conflits inutiles pendant l'apprentissage. Plus tard, tu gereras les vrais conflits. Ici, tu installes la boucle.

Regles d'equipe simples

Personne ne commit directement sur `main` : branche + PR. Messages clairs, pas "update". PR petites : une idee, quelques fichiers. `pull` le matin avant de coder. Ne jamais forcer (`push --force`) sur `main`. Ne jamais committer `.env`. Chez DanielCraft, on ecrit ces regles dans CONTRIBUTING.md avant de coder. Dix lignes. Culture visible. Lea les lit en debut de mission. Max les a ignorees une fois. Il a regrette. Sam les fait lire a voix haute.

force push ?

```
git push --force
```

A bannir sur les branches partagees tant que tu ne maitrises pas. Ca reecrit l'histoire distante. Ca peut effacer le travail des autres sans qu'ils le voient tout de suite. On le range dans "outils dangereux, pas jouets". Max l'a utilise une fois sur une branche perso. Jamais sur `main`. Lea protege `main` sur GitHub pour ne pas avoir a se faire confiance a 23h. Sam raconte l'histoire du force push "qui a tout efface" sans donner de noms. La peur utile. Pas la panique.

Protection de branche

Sur GitHub : Settings -> Branches -> proteger `main`. Exige une PR pour merger. C'est une ceinture de securite. Lea l'active meme sur ses depots solo "serieux". Bon reflexe : tu ne peux plus te merger en panic sur main sans passer par la PR. Sam le recommande des le premier projet eleve heberge sur GitHub. Max l'a active apres son incident. Mieux vaut avant.

♦ ASTUCE

Protege `main` sur GitHub (Settings -> Branches). Meme en solo, c'est une ceinture de securite qui t'oblige a passer par une PR.

Mini charte (a coller)

Personne ne pousse en force sur `main`. Toute feature passe par une PR. On review au moins 1 fois (meme soi-meme en solo, le lendemain). On ne commit pas `.env`. On ecrit des messages qui se lisent dans `log -oneline`. Colle ca dans `CONTRIBUTING.md`. Dix lignes suffisent. La culture se code aussi dans un fichier texte. Tu peux ajouter "PR petites" et "pull le matin". Lea a ajoute "PR petites" apres une review de 800 lignes qui lui a coute une soiree entiere. Regle ecrite, douleur evitee la fois suivante.

Petite histoire

Max a joué Alice/Bob un dimanche, merge deux PR, vu un historique lisible sur GitHub, sourit. Il a dit "c'est con mais ça change tout" - con en apparence, décisif en pratique. Sam a fait lire la charte à voix haute avant le premier commit en binôme. Lea a ajouté "PR petites" après sa soirée perdue. Chez DanielCraft, on aime ces règles courtes parce qu'elles tiennent. Les chartes de vingt pages, personne ne les lit. Dix lignes, oui.

Erreur classique

Travailler tous les deux sur la même ligne du même fichier "pour voir le conflit" dès la première minute - garde ça pour l'atelier conflits. Force push "parce que ça bloque". Oublier de `pull` après un merge distant. Oublier d'écrire CONTRIBUTING.md : les règles restent dans la tête, donc elles disparaissent. Autre piège : faire une seule PR monstrueuse "pour aller plus vite". Tu n'iras pas plus vite. Tu revieweras moins bien.

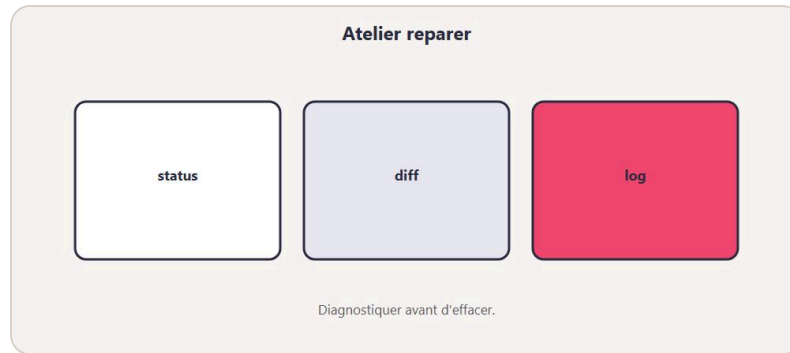
En vrai

La prochaine fois que tu travailles vraiment à deux (stage, projet associatif, client avec un dev interne), relis ta charte avant de coder. Dix lignes préviennent dix conflits évitables. Lea le fait systématiquement en début de mission. Max aussi, maintenant. Sam le fait avant chaque atelier binôme.

A toi

Joue Alice et Bob sur un dépôt de test. 2 PR mergees. Historique lisible. Écris `CONTRIBUTING.md`. Puis protège `main` si tu peux. Livrable : dépôt avec 2 PR mergees + charte + notes de ce que "Alice" a dit à "Bob" en review. Garde ce dépôt. C'est une preuve de collab autant qu'une preuve de Git.

Chapitre 19 - Atelier : reparer sans paniquer



status, diff, log : diagnostiquer avant d'effacer.

Les seniors aussi cassent. La difference : ils savent lire **status** et choisir un outil sur. Cet atelier transforme la panique en **checklist**. Chez DanielCraft, on prefere un repair lent et juste a un reset heroique. La vitesse sans lecture produit des degats. La lenteur methodique produit des projets qui tiennent. Toi, tu vas te constituer une boite a outils mentale : savoir quoi faire, dans quel ordre, selon que c'est pousse ou non.

Lea commence toujours par **status** + **log --oneline -5**. Max copie le dossier s'il doute. Sam fait casser volontairement, puis reparer, puis ecrire dans **notes.md**. Objectif : casser volontairement, reparer, documenter. Duree : 30 a 45 minutes. Ce n'est pas un permis de destruction. C'est un entrainement de pompier : tu allumes un petit feu controle pour apprendre a l'eteindre.

Panique. Stop. Status. Log. Questions : pousse ? seul ? Puis une hypothese. Une commande. Nouveau status. Si ca derape : abort, copie, aide. Chez DanielCraft, la checklist est le vrai super-pouvoir. Lea la sort avant les commandes. Max aussi, maintenant. Sam l'affiche au tableau pendant l'atelier. Toi, tu vas l'imprimer mentalement.

★ A RETENIR

En local, pas pousse : reset possible. Deja pousse / equipe : revert. Si tu hesites, copie le dossier ailleurs avant tout **--hard**.

Ce que ce n'est pas

Ce n'est pas un permis de **reset --hard** partout. Ce n'est pas non plus "Google la premiere commande et colle". Checklist d'abord. Est-ce deja pousse ? Suis-je seul sur la branche ? Alors seulement : restore / reset / revert. Ce n'est pas une humiliation si tu abort un merge. C'est de la gestion de risque. Ce n'est surtout pas "reparer sans documenter" : tu re-paniqueras la prochaine fois. Ecris. Ton **notes.md** devient ton manuel perso.

Checklist anti-stress

Commence par **git status**, puis **git log --oneline -5**. Demande-toi : est-ce deja pousse ? Suis-je seul sur la branche ? Une hypothese, une commande, un nouveau **status**. Si un conflit te depasse : **git merge --abort**, respire, reessaye plus tard. Si tu hesites sur **--hard**, copie le dossier ailleurs. La copie n'est pas une preuve de faiblesse. C'est une assurance. Max l'appelle "le parachute USB". Lea copie dans un dossier **_backup** date. Sam exige la copie avant tout marteau en atelier debutant.

Cas 1 - Mauvais fichier stage

```
git restore --staged mauvais.txt
```

Le fichier reste sur le disque. Il sort juste de l'index. Tu as retiré un objet de la table avant la photo. Pas de drame.

Cas 2 - Mauvais message (pas encore push)

```
git commit --amend -m "Meilleur message"
```

Corrige la légende de la dernière photo. Seulement si pas encore poussé / pas partagé. Sinon, discussion d'équipe. Le amend souvent en solo local. Jamais après un push partagé sans accord.

Cas 3 - Commit en trop (pas push)

```
git reset --soft HEAD~1
```

Annule le commit, garde les changements stages. Tu peux recommencer proprement. Utile quand tu as photographié trop tôt.

Cas 4 - Commit déjà push (équipe)

```
git revert HEAD  
git push
```

Nouveau commit d'annulation. Historique honnête. Préfère ça sur `main` partagé. C'est le geste adulte du chapitre 15, mis en pratique ici.

Cas 5 - Mauvaise branche

Tu as commit sur `main` au lieu de `feature` :

```
git switch -c feature/oubli  
git switch main  
git reset --hard HEAD~1  
git switch feature/oubli
```

Seulement si le commit n'est pas utile sur `main` et pas partagé. Sinon : demande conseil / fais une PR depuis ce commit autrement. Ne recopie pas cette séquence les yeux fermés. Lis la condition. Sam insiste là-dessus. Lea aussi.

Cas 6 - Conflit qui te dépasse

```
git merge --abort
```

Repars. Demande de l'aide. Réessaye plus tard. Ce n'est pas un aveu d'échec. C'est de la gestion de risque. Max abort. Lea abort. Sam célèbre l'abort. Tu peux aussi. Sam a vu un élève vouloir `--hard` tout de suite : il a fait lire la checklist d'abord. Le geste dangereux est devenu inutile. C'est le succès de l'atelier.

♦ ASTUCE

Avant tout `--hard`, dis a voix haute : "est-ce pousse ? suis-je seul ?". Si tu hesites, copie le dossier. La checklist bat le marteau.

Petite histoire

Max a amend un message non pousse, sourit. Lea a revert un commit pousse sans reecrire l'histoire des autres. Sam a vu un eleve vouloir `--hard` tout de suite : il a fait lire la checklist d'abord. Le geste dangereux est devenu inutile. Chez DanielCraft, c'est le succes de l'atelier. Pas "savoir detruire vite". Savoir choisir lentement. La prochaine panique reelle, tu sortiras la checklist avant le marteau. Tu passeras pour quelqu'un de calme. Tu le seras vraiment.

Erreur classique

Enchainer reset + amend + force push. Reparer sans savoir si c'est pousse. Oublier d'ecrire ce que tu as fait - tu re-paniqueras la prochaine fois. Documente. Ton `notes.md` devient ton manuel perso. Autre piege : reparer le symptome (message rouge) sans comprendre la cause (mauvaise branche, pull manquant, secret stage). Status d'abord. Toujours.

Exercice final atelier

Casse volontairement (mauvais add, mauvais message, petit conflit). Repare avec la methode ci-dessus. Ecris dans `notes.md` ce que tu as fait. Bonus : un cas "deja push" avec `revert`. Chronometre sans stress. Vise la clarte du compte-rendu plus que la vitesse.

En vrai

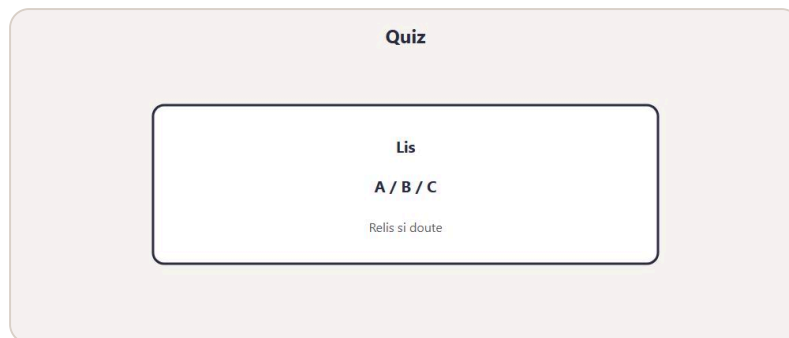
Imprime mentalement la checklist. La prochaine panique, sors-la avant les commandes. Dis-la a voix haute si besoin. Lea le fait encore parfois a 23h. Max aussi. Sam le recommande sans ironie. Le rituel bat l'ego.

A toi

Fais l'exercice final. Puis explique a voix haute quand tu choisis `restore`, `reset`, ou `revert`. Si tu distingues les trois sans notes, tu as gagne l'atelier. Garde ton `notes.md` : c'est ton diplome anti-panic.

QUIZ

Quiz final - Teste-toi !



Vérifier sans inventer.

Pas de piège. L'idée : vérifier que tu pilotes Git avec méthode - **zones**, **branches**, **remote**, **conflits**, **undo**, **PR**. Chez DanielCraft, un quiz sert à repérer les trous, pas à humilier. Lea note ses erreurs et rouvre deux chapitres. Max refait le mini-projet. Sam utilise ce QCM en fin de module. Toi, tu vas répondre sans regarder. Puis seulement, comparer. Si tu bloques sur une question, note le numéro : c'est ton plan de relecture. Pas besoin de tout relire. Juste les trous.

Réponds au calme. Une question à la fois. Si une réponse te paraît "trop évidente", c'est souvent la bonne : on ne cherche pas à te piéger avec des subtilités de rebase interactif. On cherche à savoir si tu as le sol. Le sol, c'est ce qui te sauve un vendredi soir. Chez DanielCraft, le vrai score, c'est la deuxième passe une semaine plus tard. La mémoire se teste dans le temps, pas seulement le jour même.

Tu es devant un terminal rouge un jour de livraison. Tu ne recites pas vingt flags. Tu te demandes : **status** ? **pousse** ? **branche** ? **secret** ? **conflit** ? **PR** ? Ce quiz vérifie que ces questions te viennent. Si oui, tu pilotes. Si non, tu sais où revenir. C'est déjà beaucoup. Chez DanielCraft, piloter bat reciter.

♦ ASTUCE

Refais ce quiz dans une semaine, sans relire le livre. Le vrai score, c'est la deuxième passe. Sam le sait : la mémoire se teste dans le temps, pas le jour même.

Ce que ce n'est pas

Ce n'est pas un examen universitaire. Ce n'est pas non plus "tu as rate donc tu es nul en Git". Une question ratée pointe un chapitre. Tu rouvres dix minutes. Tu retestes demain. Ce n'est pas une course. Dix-huit questions, du temps, des notes sur papier. Lea a raté les questions **undo** la première fois. Elle a rouvert le chapitre 15. Elle a réussi la semaine suivante. Max aussi. Sam adore ces trajectoires.

Questions

1. 1. Git sert surtout à :

- A) Dessiner des maquettes
- B) Garder l'historique des versions
- C) Remplacer un antivirus

1. 2. GitHub, c'est :

- A) Un hebergeur de depots Git (+ collab)
- B) Un langage
- C) Un fichier `.gitignore`

1. 3. `git init` :

- A) Envoie sur internet
- B) Cree un depot local
- C) Supprime l'historique

1. 4. Avant `commit` , on fait souvent :

- A) `git add`
- B) `git push`
- C) `git clone`

1. 5. `git status` sert a :

- A) Voir l'etat du dossier / index
- B) Fusionner
- C) Creer une PR

1. 6. Une branche permet de :

- A) Compresser les images
- B) Travailler en parallele sans casser main
- C) Effacer GitHub

1. 7. `git merge` :

- A) Clone un depot
- B) Fusionne des historiques / branches
- C) Ignore les fichiers

1. 8. Un conflit veut dire :

- A) Git a plante pour toujours
- B) Deux versions du meme endroit a arbitrer
- C) Le disque est plein

1. 9. `.gitignore` sert a :

- A) Decoruer le README
- B) Exclure des fichiers du suivi
- C) Accelerer internet

1. 10. `git push` :

- A) Envoie tes commits vers le remote
- B) Telecharge seulement
- C) Cree une branche locale

1. 11. `git pull` :

- A) Supprime origin
- B) Recupere (et integre) les commits distants
- C) Remplace `commit`

1. 12. Un bon message de commit :

- A) `update`
- B) `Corriger le lien casse du menu`
- C) `asdf`

1. 13. `git stash` :

- A) Met des modifs de cote temporairement
- B) Pousse sur GitHub
- C) Cree un remote

1. 14. Sur un commit deja partage en equipe, prefere :

- A) `reset --hard` silencieux
- B) `git revert`
- C) Effacer le depot

1. 15. Une pull request, c'est :

- A) Demander a fusionner une branche (souvent via review)
- B) Un virus
- C) Un type de conflit

1. 16. `git clone` :

- A) Copie un depot distant en local
- B) Efface main
- C) Remplace status

1. 17. `git log --oneline` :

- A) Affiche l'historique compact
- B) Installe Git
- C) Ouvre GitHub

1. 18. Si push est rejected car remote en avance :

- A) Ignorer
- B) `git pull` puis reparer eventuellement, puis push
- C) Formater le PC

Petite histoire

Lea a fait ce quiz le soir du mini-projet : 15/18. Elle a noté les trois erreurs (stash, revert, PR description mentale). Elle a rouvert deux chapitres dix minutes. Le lendemain, ces trois questions étaient solides. Max a eu 12/18 la première fois, 17/18 une semaine plus tard sans relire le livre entier. Sam utilise exactement ce déroulement en classe. Chez DanielCraft, on célèbre la courbe, pas le score du jour 1.

Erreur classique

Relire les réponses en même temps que les questions "pour aller plus vite". Chercher la réponse dans le livre pendant le quiz. Se juger sur un mauvais score sans plan de relecture. Autre piège : réussir 18/18 le jour même et ne jamais retester - la mémoire se teste dans le temps. Planifie J+7. Cinq minutes. Grand effet.

★ A RETENIR

Note les numéros ratés. Rouvre les chapitres. Reteste demain. Le vrai score, c'est la deuxième passe.

Reponses

1 B, 2 A, 3 B, 4 A, 5 A, 6 B, 7 B, 8 B, 9 B, 10 A, 11 B, 12 B, 13 A, 14 B, 15 A, 16 A, 17 A, 18 B

Score

16-18 : nickel. 12-15 : bien, relis conflits + undo + PR. 8-11 : refais 4, 6, 8, 11, 14. Moins de 8 : refais le mini-projet tranquillement. Puis rejoue le quiz dans une semaine. Chez DanielCraft, le vrai score, c'est la deuxième passe. Lea colle son score sur un post-it avec la date. Max aussi. Sam collecte les courbes anonymisées pour ajuster ses ateliers.

En vrai

Note les numéros des questions ratées. Rouvre les chapitres correspondants dix minutes chacun. Puis refais seulement ces questions demain, sans relire le reste. C'est plus efficace qu'une relecture linéaire du livre entier. Tu cibles. Tu consolides. Tu avances.

A toi

Reponds au quiz sans notes. Corrige. Ecris en trois lignes : (1) ta meilleure réponse, (2) la question qui t'a surpris, (3) le chapitre que tu rouvriras en premier. Garde ce papier pour la deuxième passe dans une semaine. Puis accepte le bravo du chapitre suivant - tu l'as mérité, même avec des trous. Les trous font partie du chemin.

Bravo.

Bravo

Tu as les bases de Git et GitHub

Maintenant : commit, branche, push.

Bravo. Tu as les bases de Git et GitHub.

Tu as fini les bases de Git et GitHub. Pas "je connais toutes les options cachees". Mais "je sais **versionner**, **brancher**, **fusionner**, **pousser**, et me sortir d'un **conflit**". C'est le coeur du metier. Chez DanielCraft, on considere que c'est ca, la vraie competence debutant solide en 2026. Pas reciter vingt flags. Piloter un depot avec methode. Tu n'es pas expert en rebase interactif. Tu n'as pas besoin de l'etre demain matin. Tu es quelqu'un qui sait allumer la machine a photos, lire status, oser une branche, joindre, pousser, arbitrer. C'est deja beaucoup. C'est surtout actionnable.

Lea repart avec des PR propres et un `.gitignore` reflexe. Max repart sans peur des conflits. Sam repart avec des ateliers rejouables en classe. Toi, tu repars avec une memoire pour ton travail - et une checklist anti-panique. Garde `mes-tests-git` ou `carnet-git`. Ce n'est pas un jouet fini. C'est un gymnase. Tu y reviendras quand tu apprendras un nouveau langage ou un nouveau framework. Git ne change pas. Tes projets, si.

★ A RETENIR

Git photographie. Toi, tu choisis quoi garder. Garde le gouvernail. Encore bravo. Ton code a maintenant une memoire. Et toi aussi.

Ce que ce n'est pas

Ce bravo, ce n'est pas "tu n'auras plus jamais de message rouge". Tu en auras. La difference : tu sauras les lire. Ce n'est pas non plus "Git c'est fini, on n'en parle plus". Le muscle se perd sans pratique. Ce n'est pas un diplome officiel. C'est une competence posee. Chez DanielCraft, on prefere une competence posee a un certificat oublie. Lea le sait. Max aussi. Sam le rappelle a chaque promo.

Ce que tu sais faire

Tu sais installer et configurer Git. Tu utilises `add`, `commit`, `log` et `status`. Tu geres branches, merge et conflits. Tu ecris un `.gitignore`. Tu manipules remote, clone, push et pull. Tu te debrouilles avec stash et les annulations raisonnables. Et tu fais des pull requests. Tu sais aussi ce que tu ne sais pas encore : rebase avance, hooks, monorepos, CI complexe. C'est sain. La carte mentale inclut les zones grises. Tu n'as pas peur d'y aller plus tard, avec prudence.

Mission finale

Sur un vrai mini projet (meme un site de 2 pages), cree un depot GitHub. Travaille sur une branche feature. Merge via une PR. Ecris un README clair. Verifie qu'aucun secret n'est dans l'historique. Si tu codes aussi en Python, versionne ton quiz ou ton juste prix : les deux livres se parlent bien. Chez DanielCraft, on aime ces ponts entre formations. Git est le sol commun.

La suite (quand tu voudras)

Tu pourras regarder les hooks / pre-commit, le rebase interactif (avec prudence), les monorepos, GitHub Actions (CI), et des conventions d'equipe plus fines. Mais la, respire. Une habitude posee bat dix tutos binge. Lea bingeait trop au debut. Elle a gagne plus en commitant chaque jour qu'en regardant trois heures de videos. Max aussi. Sam l'enseigne sans juger le binge - juste en proposant mieux.

Rituel des 2 minutes

Avant chaque session :

```
git status
git pull
```

Apres chaque morceau utile :

```
git status
git add .
git commit -m "...
git push
```

Regarde toujours `status` avant `add .`. Les habitudes battent la memoire. Colle ce rituel sur un post-it une semaine. Ensuite, c'est dans les doigts. Lea n'y pense plus. Max y pense encore le lundi. Sam le fait en demo. Toi, choisis un moment fixe : debut de session. Fin de session. Les deux, c'est encore mieux.

Engagement de 30 jours

Chaque jour de travail : au moins un commit utile avec un message clair. Chaque semaine : une branche + une PR (meme solo). J+30 : refaire le quiz et relire ta checklist anti-stress. Tu transformeras ce livre en muscle. C'est le vrai bravo. Pas la derniere page. Les trente jours qui suivent. Chez DanielCraft, on mesure le succes au J+30 plus qu'au "j'ai fini le PDF".

♦ ASTUCE

Bloque dix minutes dans ton agenda a J+7 (quiz) et J+30 (quiz + checklist). Deux rappels. Grand effet sur la memoire.

Petite histoire

Lea a montre son depot `carnet-git` a un client : le README etait clair, l'historique lisible. Le client n'a pas compris Git, mais il a compris que son projet etait entre de bonnes mains. Max a refait le quiz une semaine plus tard : 17/18. Sam a deja planifie de rejouer les ateliers avec sa prochaine promo. Chez DanielCraft, c'est le signe que le livre a fait son travail. Pas "tout su". "Assez su pour continuer seul, avec methode".

Erreur classique

Croire que "j'ai fini le livre" egal "je n'aurai plus jamais de message rouge". Non. Tu auras des messages rouges. La difference : tu sauras les lire. Autre piege : ne plus jamais committer parce que "Git c'est fini". Le muscle se perd sans pratique. Encore un piege : vouloir tout apprendre (rebase, hooks, submodules) la semaine suivante sans solidifier le sol. Monte un etage. Pas dix d'un coup.

En vrai

Garde le dossier `mes-tests-git` ou `carnet-git`. Reviens-y quand tu apprends un nouveau langage ou un nouveau framework. Git ne change pas. Tes projets, si. Ouvre `status`. Tire. Commit. Pousse. Sens que tu pilotes. C'est ton rituel de remise en main. Si un message te bloque, relis-le jusqu'au bout avant de chercher un tutoriel : souvent la reponse est deja la. Chez DanielCraft, c'est le reflexe qui reste apres le livre.

A toi

Accepte l'engagement de 30 jours ci-dessous. Un commit utile par jour de travail. Une branche + PR par semaine. J+30 : refais le quiz et relis ta checklist anti-stress. Ecris aussi en une phrase ce que Git a change dans ta facon de travailler. Garde-la. Relis-la dans un mois. Encore bravo. Vraiment.

Tu as les briques. Maintenant, construis.